Declaration of Original Work for SC2006 Assignment

We hereby declare that the attached group assignment has been researched, undertaken, completed, and submitted as a collective effort by the group members listed below.

We have honored the principles of academic integrity and have upheld Student Code of Academic Conduct in the completion of this work.

We understand that if plagiarism is found in the assignment, then lower marks or no marks will be awarded for the assessed work. In addition, disciplinary actions may be taken.

Name	**Course**	**Lab Group**
Toh Kok Soon	CSC	SCS7
Murali Iniya	CSC	SCS7
Prakriti Muralidharan	CSC	SCS7
Murugesan Chandramohan	CSC	SCS7

Important notes:

1. Name must EXACTLY MATCH the one printed on your Matriculation Card.
2. Student Code of Academic Conduct includes the latest guidelines on usage of Generative AI and any other guidelines as released by NTU.

Project Title: CommuteBuddy

Group Members:

Name	**Student ID**
Toh Kok Soon	U2423531K
Murali Iniya	U2421833K
Prakriti Muralidharan	U2422005C
Murugesan Chandramohan	U2420232A

1. Purpose

1.1. Project Mission Statement

To improve the convenience and efficiency of Singapore's transport systems through the use of data that helps commuters make better-informed decisions.

Primary Target Audience:

- Daily Commuters (students, office workers, drivers, etc.)

Secondary Target Audience:

- Carpark Managers (HDB, LTA, URA, etc.)

2. Requirement Elicitation

2.1. Legacy Product Study

2.1.1. Carpark Availability Apps

Primitive carpark availability apps tend to use a list to represent carparks instead of doing it on a map, which makes it hard for users to get an overview.

2.2. Competitive Product Study

2.2.1. carparksg.com

Not many car park availability apps are available, especially on the Google Play Store. The most similar product to our CommuteBuddy carpark availability system that we have identified is a web application found on the site carparksg.com (*Real-time carpark availability in Singapore, n.d.*).

Based on our experiences with the application, we have taken note of the following:

Good	Bad

• Map view enables smooth navigation, such as zooming on clusters of parking lots by clicking on the cluster icon.
• Information is abstracted when zoomed out, therefore improving readability and avoiding overwhelming users with dense information.
• The app works on both PC and mobile.
	• UI is not straight away intuitive:
	  ○ More lots are red and fewer lots are blue.
	  ○ Red markers for carpark with lots and grey markers for carpark with no lots remaining.
	• The numbers shown show how many carparks there are in the vicinity, but do not show lots availability.

	● No way to add carparks outside of their limited data. ● Users can favourite carparks, but the data is not tied to a user account.

2.2.2. Google Maps

Most of the core functionality planned for our public transport planning system has already been covered by Google Maps. What separates us from Google Maps is our focus on a transport app that also includes carpark availability. Therefore, we are able to cater our UI to be more friendly towards our target audience. Furthermore, it would also be a more convenient app for commuters that both use public transport and drive.

2.3. Functional Requirements

1. The system shall allow users to manage their accounts.
    - 1.1. The system shall allow users to register an account using email and password.
    - 1.2. The system shall allow users to log in securely with registered credentials.
    - 1.3. The system shall store user data in a database to ensure synchronization across devices.
    - 1.4. The system shall allow user to change their account detail
    - 1.5. The system shall allow user to change their password when they forget password
    - 1.6. The system shall use hash to store password for security reasons
2. The system shall allow users to search for and manage carparks.
    - 2.1. The system shall allow users to view real-time carpark availability on a map.
        - 2.1.1. The map shall display the number of available lots using a marker
        - 2.1.2. Map marker should include the number of lots available
        - 2.1.3. Map marker should be color coded to show percentage of lots available
            - 2.1.3.1. Red for 25% lots left
            - 2.1.3.2. Yellow for 50% lots left
            - 2.1.3.3. Green for 75% lots left
        - 2.1.4. The map marker should not overlap each other, lots in the vicinity should be grouped into a single marker when zoomed out
    - 2.2. The system shall allow users to view detailed information about individual carparks whenever data is available
        - 2.2.1. Detailed information can be viewed by clicking on the map marker of the individual carpark
        - 2.2.2. Detailed information include gantry heights
    - 2.3. The system shall allow users to bookmark frequently accessed carparks.
    - 2.4. The system shall allow users to filter carparks shown on the map by:
        - 2.4.1. Favourited status
        - 2.4.2. Gantry height
        - 2.4.3. Distance radius

2.5. The system shall allow carpark managers to publish carpark information (e.g., location, lot availability, gantry height) through Carpark APIs.
2.6. The system shall allow administrators to approve or reject Carpark API key requests from carpark managers.
2.7. The system shall allow user-added carparks to be tested using simulated availability data.

3. The system shall provide public transport planning features.
3.1. The system shall allow users to search and check real-time arrival times for buses.
3.2. The system allows users to favourite their selected bus stops in the user's account. The users can also delete and edit their favourites.

4. The system shall provide meaningful error feedback.
4.1. There should be a designated area to display error messages on forms.
4.2. Error messages should appear as pop-ups, wherever no designated area is assigned for the page.

2.4. Non-Functional Requirements

Usability	New User Learnability and Simplicity ● The user interface shall be simple and intuitive.
	Readable Interface ● Information displayed shall not be overly dense, and text or numbers shall not overlap.
	Platform Responsive ● Users must be able to see 100% of the content regardless of what platform they are using. ● No content shall be clipped or hidden away due to mobile size issues.
Performance	Data Retrieval Efficiency ● The system shall retrieve and display real-time bus arrival data within 3 seconds. ● The system shall update carpark availability data at least every 60 seconds
Reliability	Quick App Startup ● In the event of a system crash or reboot, the system shall restore full functionality within 5 minutes.

Supportability	Accessibility ● The system shall be cross-platform and accessible on: ○ Mobile (iOS, Android) ○ Tablets ○ PCs via web interface

2.5. Data Dictionary

2.5.1. Non-Database Data Dictionary

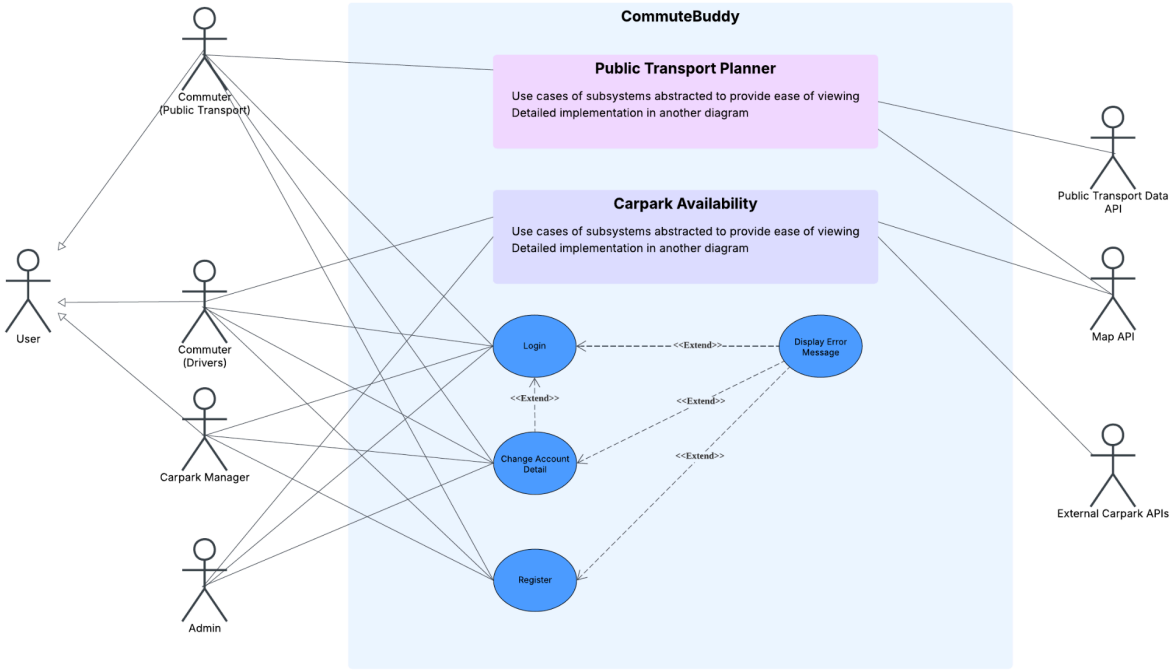
Term	Synonyms	Definition
Application	CommuteBuddy WebApp App Software	The software solution proposed by us
Users		Individuals who interact with the application, including commuters, admins, carpark managers, and urban developers.
User Account		User details needed for authentication, and their credentials
User Role	User Type	Classification of User (e.g., commuters, carpark managers)
User Profile		A collection containing user preferences (e.g., preferences).
Commuters	Travellers	The general public that uses transport infrastructures
Admins		Authorized staff who oversee and maintain the application operation
Carpark Manager		Person, Organisation, or Agencies in charge of one or more carparks
Carpark	Parking	A designated parking facility for vehicles.
Carpark Source		Source of information for carpark data
Carpark Availability		The current number of parking lots available at a car park. (i.e., high availability means more free lots)

Carpark Utilisation		Percentage of carpark lots being used up (i.e. high utilisation means fewer lots available)
Gantry Height		The maximum vehicle height allowed at the entrance of a car park.
Carpark Preference Filter		A user-defined criterion for filtering carparks (e.g., closest distance, gantry height suitable for the vehicle).
Bookmark	Favourite	A feature allowing the saving of an in-app data point
Bus Service		A scheduled bus line, identified by its service number (e.g., 7, 106, 961).
Bus Stop		A designated location where passengers can board or alight from buses.
MRT Line	Train Line	A railway line in Singapore's MRT network (e.g., East-West Line, Circle Line) can also be identified by a 2-letter acronym (e.g., CC, NS, EW)
MRT Station	Train Station	A designated train stop where commuters can board or alight, represented by a 2-letter acronym followed by a number (e.g. NS1, EW21), a train station may include one or more such representations
Bus Interchange		A central transport hub connecting multiple bus services, often connected to a train station
Train Interchange		A train station connecting more than one train line
Mode of Transport		The type of travel chosen by the commuter (e.g., MRT, bus, walking, private car).
Estimated Travel Time (ETT)	Estimated Time of Arrival (ETA)	The predicted duration of the arrival timing of public transport
Crowd Density		The general crowd level (e.g. low, medium and high) in an area

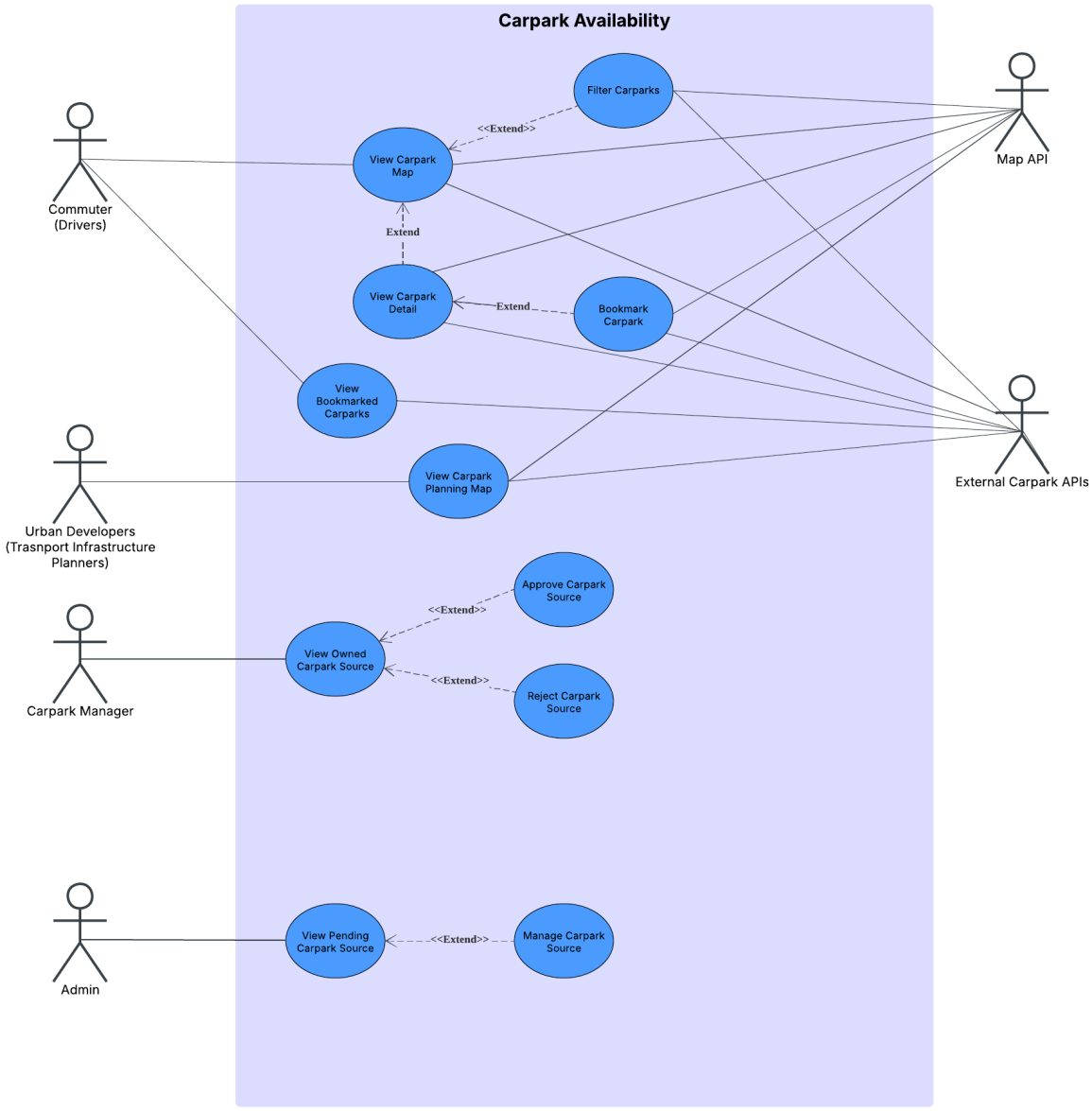
3. Prototype

3.1. Use Case Diagrams

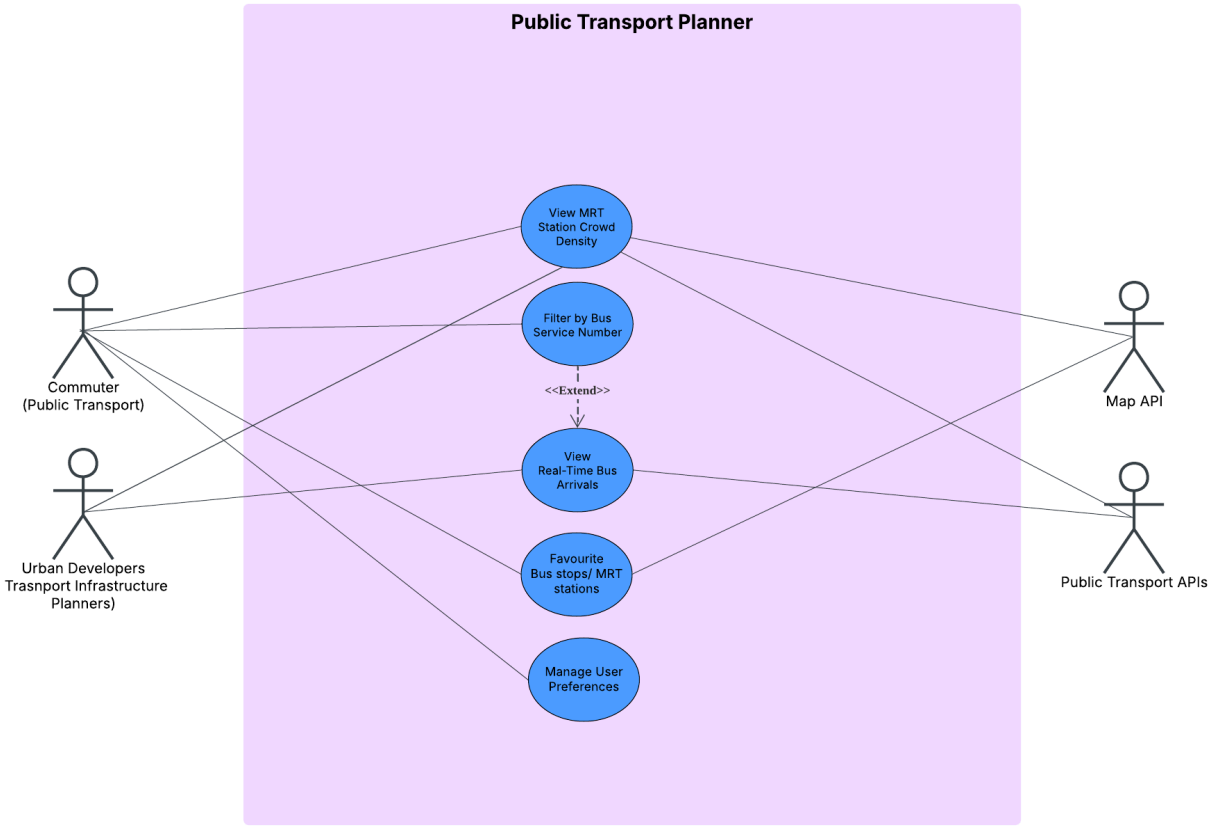
3.1.1. General Overview



3.1.2. Carpark Availability System



3.1.3. Public Transport Planner System



3.2. Use Case Descriptions

3.2.1. Functional Requirement #1: Account Management

3.2.1.1 Login

Use Case ID:	UC1		
Use Case Name:	Login		
Created By:	Toh Kok Soon	Last Updated By:	Toh Kok Soon
Date Created:	01/09/2025	Date Last Updated:	01/09/2025
Actor	User		
Description:	Allows the app to identify the user from the details they entered		
Preconditions:	• User has an account		
Postconditions:	• Application saves the logged-in state of the user		
Priority:	High		
Frequency of Use:	Medium to High		
Flow of Events:	1. User enters email and password, and then presses login 2. The format of the email is checked 3. The entered detail is passed to the application backend for authentication 4. Look for the username in the database and check its corresponding password hash for a match 5. Sends the user back to the page they were on		
Alternative Flows:	AF-S1: 1. If user click “forgot password”, goes to use case 3 (Change Account Detail) AF-S2: 1. If the format of the email is wrong, the error message “Invalid Email Format” is displayed on the login screen AF-S4: 1. If an account with the email is not found or the password does not match 2. The error message “Invalid login information" is displayed on the login screen		
Exceptions:	EX1: email format is wrong → AF-S2 EX2: user account not found → AF-S4		
Includes:	-		
Special Requirements:	-		
Assumptions:	• Database connection available		
Notes and Issues:	The invalid login information message is deliberately kept vague for security		

	reasons

3.2.1.2 Register

Use Case ID:	UC2		
Use Case Name:	Register		
Created By:	Toh Kok Soon	Last Updated By:	Toh Kok Soon
Date Created:	01/09/2025	Date Last Updated:	01/09/2025
Actor	User (except Admin)		
Description:	Allows the user to create an account		
Preconditions:	• User has no account		
Postconditions:	• Account is created and saved in the database		
Priority:	Medium		
Frequency of Use:	Low		
Flow of Events:	1. User fill up registration form: a. User enter their nickname b. User re-type email c. User enters password d. User re-type password e. User press register 2. Input is validated to ensure correctness 3. User account is created in the database		
Alternative Flows:	<u>AF-S2:</u> 1. If input validation fails, collect all errors 2. Display error messages on screen and wait for user to click ok 3. Goes back to step 1 of main flow		
Exceptions:	EX1: invalid nickname format → AF-S2 EX2: invalid email format → AF-S2 EX3: email already in used → AF-S2 EX4: email does not match → AF-S2 EX5: password does not meet the requirement → AF-S2 EX6: password does not match → AF-S2		
Includes:	-		
Special Requirements:	-		
Assumptions:	-		

Notes and Issues:	-

3.2.1.3 Change Account Detail

Use Case ID:	UC3		
Use Case Name:	Change Account Detail		
Created By:	Toh Kok Soon	Last Updated By:	Toh Kok Soon
Date Created:	01/09/2025	Date Last Updated:	01/09/2025
Actor	User		
Description:	Allows the user to change their account credentials		
Preconditions:	● User has an account		
Postconditions:	● Account details changed are saved in the database		
Priority:	Medium		
Frequency of Use:	Low		
Flow of Events:	1. User goes to the profile page 2. They can enter a new email or a new password and click update 3. The details are saved in the database		
Alternative Flows:	<u>AF-S1:</u> 1. If user is not logged in, they can change their password by clicking “forgot password” from login page 2. User have to enter their email 3. A random password will be generated and saved into the database for the account with the email 4. An email with the randomly generated password will be sent to the user email <u>AF-S2:</u> 1. If the email they entered is already in use, display error message “email already in use” 2. skip step 3		
Exceptions:	EX1: invalid email format → user have to re-enter their email EX2: user account with the email address is not found → display error message “user account not found” EX3: email already in use → AF-S2		
Includes:	-		
Special Requirements:	-		
Assumptions:	● Database connection is available		

Notes and Issues:	-

3.2.2. Functional Requirement #2: Searching and Managing of Carparks

3.2.2.1 Add Carpark Source

Use Case ID:	UC4		
Use Case Name:	Add carpark source		
Created By:	Toh Kok Soon	Last Updated By:	Toh Kok Soon
Date Created:	01/09/2025	Date Last Updated:	23/10/2025
Actor	Carpark Manager		
Description:	Allow user to publish a source for carpark data		
Preconditions:	● User is logged in		
Postconditions:	● API key request is saved in the database		
Priority:	Medium		
Frequency of Use:	Low		
Flow of Events:	1. User goes to settings 2. User clicks “manage my carpark sources” 3. User clicks on “add carpark sources” 4. A form will appear for user to fill up: a. User inputs API endpoints for carpark availability and carpark info b. User specify any necessary headers (api key etc.) c. User specify field mappings 5. Create a database entry with the status as pending		
Alternative Flows:	-		
Exceptions:	-		
Includes:	-		
Special Requirements:	-		
Assumptions:	● Database connection is available		
Notes and Issues:	-		

3.2.2.2 Remove Carpark Source

Use Case ID:	UC5		
Use Case Name:	Remove Carpark Source		
Created By:	Toh Kok Soon	Last Updated By:	Toh Kok Soon
Date Created:	01/09/2025	Date Last Updated:	23/10/2025
Actor	Carpark Manager		
Description:	Allows user to delete an existing carpark source		
Preconditions:	● User is logged in ● User has carpark source		
Postconditions:	● carpark source is removed from the database		
Priority:	Low		
Frequency of Use:	Low		
Flow of Events:	1. User goes to “settings” 2. User clicks on “manage my carpark sources” 3. User clicks “remove carparks source” on the carpark source that they want to remove 4. Look for the carpark source in the database 5. Remove carpark source from database		
Alternative Flows:	-		
Exceptions:	-		
Includes:	-		
Special Requirements:	● Database connection is available		
Assumptions:	-		
Notes and Issues:	-		

3.2.2.3 Manage Carpark Source

Use Case ID:	UC6		
Use Case Name:	Manage Carpark Source		
Created By:	Toh Kok Soon	Last Updated By:	Toh Kok Soon
Date Created:	01/09/2025	Date Last Updated:	23/10/2025

Actor	Admin
Description:	A collection of actions used by the admin involving API Keys status
Preconditions:	• User is logged in • User is admin
Postconditions:	-
Priority:	Low
Frequency of Use:	Low
Flow of Events:	1. Admin can select Carpark Source from a list to see its detail 2. Admin can add,remove or delete existing Carpark Sources
Alternative Flows:	-
Exceptions:	-
Includes:	-
Special Requirements:	-
Assumptions:	-
Notes and Issues:	-

3.2.2.4 View Carpark Map

Use Case ID:	UC7		
Use Case Name:	View Carpark Map		
Created By:	Toh Kok Soon	Last Updated By:	Toh Kok Soon
Date Created:	02/09/2025	Date Last Updated:	02/09/2025
Actor	Commuter (Drivers), Map API, External Carpark API		
Description:	Visual representation of carpark availability		
Preconditions:	-		
Postconditions:	-		
Priority:	Medium		
Frequency of Use:	High		
Flow of Events:	1. Checks if the user has geolocation permission on 2. Get carpark data from the database (see notes for details)		

	3. Render map with carpark data through Maps API 4. Centers map on user's current location
Alternative Flows:	AF-S1: 1. Ask for the user's geolocation permission AF-S4 1. If unable to access user current location, centers on a default location
Exceptions:	EX1: Map API not responding → Display an error message "No Map Data available" EX2: Unable to access user geolocation → AF-S4 EX3: External Carpark API not responding → Display an error message "No Carpark Data Available" and render an empty map EX4: External Carpark API returning wrong format → Ignore all the invalid data, only show the data that comes back in the right format, send an email to the owner of the API to inform them about the wrong format
Includes:	-
Special Requirements:	● Map marker should be color coded to reflect percentage of lots available ● Map marker should include number of lots available
Assumptions:	● Map API and Carpark API connections are available
Notes and Issues:	Carpark data, which includes availability of lots and location coordinates will be updated periodically in the database through API calls to external carpark APIs

3.2.2.5 Filter Carparks by Attributes

Use Case ID:	UC8		
Use Case Name:	Filter Carpark by Attributes		
Created By:	Toh Kok Soon	Last Updated By:	Toh Kok Soon
Date Created:	02/09/2025	Date Last Updated:	02/09/2025
Actor	Commuter (Drivers), Map API, External Carpark API		
Description:	Allows the user to select the kind of car park that is shown on the map		
Preconditions:	-		
Postconditions:	-		
Priority:	Medium		
Frequency of Use:	High		

Flow of Events:	1. The user can select from various dropdown menus, such as gantry height and carpark type 2. The program takes in the user preferences and renders them on the map using the Map API
Alternative Flows:	-
Exceptions:	EX1: Map API not responding → Display an error message “No Map Data available” EX2: External Carpark API not responding → Display an error message “No Carpark Data Available” and render an empty map EX3: External Carpark API returning wrong format → Ignore all the invalid data, only show the data that comes back in the right format, send an email to the owner of the API to inform them about the wrong format
Includes:	-
Special Requirements:	-
Assumptions:	● Map API and Carpark API connections are available
Notes and Issues:	-

3.2.2.6 View Carpark Detail

Use Case ID:	UC9		
Use Case Name:	View Carpark Detail		
Created By:	Toh Kok Soon	Last Updated By:	Toh Kok Soon
Date Created:	02/09/2025	Date Last Updated:	02/09/2025
Actor	Commuter (Drivers), Map API, External Carpark API		
Description:	Show detailed information about the individual carparks		
Preconditions:	-		
Postconditions:	-		
Priority:	Medium		
Frequency of Use:	Medium		
Flow of Events:	1. The user clicks on the marker representing the car park on the map 2. A small pop-up window will list out all the details of the car park		
Alternative Flows:	<u>AF-S1:</u> 1. If the marker the user clicks represents multiple carparks in the		

	vicinity, zoom the map in to display more markers
Exceptions:	EX1: Map API not responding → Display an error message “No Map Data available” EX2: External Carpark API not responding → Display an error message “No Carpark Data Available” and render an empty map EX3: External Carpark API returning wrong format → Ignore all the invalid data, only show the data that comes back in the right format, send an email to the owner of the API to inform them about the wrong format
Includes:	-
Special Requirements:	-
Assumptions:	• The map marker the user clicked on represent an individual carpark → AF-S1
Notes and Issues:	-

3.2.2.7 Bookmark Carpark

Use Case ID:	UC10		
Use Case Name:	Bookmark Carpark		
Created By:	Toh Kok Soon	Last Updated By:	Toh Kok Soon
Date Created:	02/09/2025	Date Last Updated:	02/09/2025
Actor	Commuter (Drivers), Map API, External Carpark API		
Description:	Allow the user to save carparks that are frequently accessed		
Preconditions:	User is logged in		
Postconditions:	Bookmarked carparks are saved in the database		
Priority:	Low		
Frequency of Use:	Medium		
Flow of Events:	1. User accesses the individual carpark through the “view carpark detail” use case 2. User clicks on the bookmark to save the carpark		
Alternative Flows:	-		
Exceptions:	EX1: Map API not responding → Display an error message “No Map Data available” EX2: External Carpark API not responding → Display an error message “No		

	Carpark Data Available” EX3: External Carpark API returning wrong format → Ignore all the invalid data, only show the data that comes back in the right format, send an email to the owner of the API to inform them about the wrong format
Includes:	-
Special Requirements:	-
Assumptions:	● Map API and Carpark API connections are available
Notes and Issues:	-

3.2.2.8 View Bookmarked Carparks

Use Case ID:	UC11		
Use Case Name:	View Bookmarked Carparks		
Created By:	Toh Kok Soon	Last Updated By:	Toh Kok Soon
Date Created:	02/09/2025	Date Last Updated:	02/09/2025
Actor	Commuter (Drivers), External Carpark API		
Description:	A list view of all the user car parks		
Preconditions:	User is logged in		
Postconditions:	-		
Priority:	Low		
Frequency of Use:	Medium		
Flow of Events:	1. User enters “bookmarked carparks” 2. The program gathers bookmarked car parks from the database 3. The program then populates the bookmarked carpark with its detailed information gathered from the External Carpark API into a list view		
Alternative Flows:	-		
Exceptions:	EX1: No bookmarked carparks → Show empty list EX2: External Carpark API not responding → Display an error message “No Carpark Data Available” EX3: External Carpark API returning wrong format → Ignore all the invalid data, only show the data that comes back in the right format, send an email to the owner of the API to inform them about the wrong format		

Includes:	-
Special Requirements:	-
Assumptions:	● Carpark API connections are available
Notes and Issues:	-

3.2.3. Functional Requirement #3: Public Transport Planning Features

3.2.3.1 Favourite bus stops, MRT stations

Use Case ID:	UC12		
Use Case Name:	Favourite bus stops, MRT stations		
Created By:	Prakriti Muralidharan	Last Updated By:	Prakriti Muralidharan
Date Created:	02/09/2025	Date Last Updated:	23/10/2025
Actor	Commuter (public transport)		
Description:	Allows the user to save frequently used bus stops and MRT stations		
Preconditions:	● User is logged in		
Postconditions:	● Favourite bus stops/MRT stations are stored under the user's profile		
Priority:	Medium		
Frequency of Use:	Frequent (daily commuters)		
Flow of Events:	1. User searches/ selects a bus stop/ MRT station 2. User favourites it 3. The route is saved for future use/ quick access under the Bookmarks section		
Alternative Flows:	AF-S2: 1. If the bus stop/ MRT station is already bookmarked, unbookmark it 2. Skip step 3		
Exceptions:	EX1: Storage/ database error		
Includes:	-		
Special Requirements:	Cloud sync with the account		

Assumptions:	• User accounts support these features
Notes and Issues:	Manage storage for multiple favourites

3.2.3.2 Search and view Real-time bus arrival times

Use Case ID:	UC13		
Use Case Name:	Search and view Real-time bus arrival times		
Created By:	Prakriti Muralidharan	Last Updated By:	Prakriti Muralidharan
Date Created:	23/10/2025	Date Last Updated:	23/10/2025
Actor	Commuter (public transport)		
Description:	Allows the user to view real-time arrival information for buses at the stop.		
Preconditions:	• User is logged in • User keys in the bus stop code (valid)		
Postconditions:	• Real-time bus arrival information is displayed		
Priority:	High		
Frequency of Use:	Very Frequent (multiple times per day for regular commuters)		
Flow of Events:	1. User opens the public transport planning feature 2. User enters bus stop name, code, or selects from nearby stops 3. System retrieves bus stop information from the transport API System displays a list of bus services at that stop 4. The system shows real-time arrival times for each bus service 5. The system continuously updates arrival times every 20 seconds		
Alternative Flows:	AF-S3: Use Current Location 1. User selects "Find Nearby Stops" option 2. System requests location permission (if not already granted) 3. System displays nearest bus stops within 500m radius 4. User selects desired stop 5. Continue from step 4 of main flow		
Exceptions:	EX1: Network error EX2: Invalid bus stop code/ number EX3: Transport API unavailable		

Includes:	-
Special Requirements:	-
Assumptions:	● User accounts support these features
Notes and Issues:	Manage storage for multiple favourites

3.2.3.3 Search and view MRT station crowd density

Use Case ID:	UC14		
Use Case Name:	Search and view MRT station crowd density		
Created By:	Prakriti Muralidharan	Last Updated By:	Prakriti Muralidharan
Date Created:	23/10/2025	Date Last Updated:	23/10/2025
Actor	Commuter (public transport)		
Description:	Allows the user to search for MRT station by name and view real-time crowd density information to help plan their travel during less crowded period		
Preconditions:	● User is logged in ● User keys in the MRT name (valid)		
Postconditions:	● MRT station crowd density is displayed (Low, Medium, High)		
Priority:	High		
Frequency of Use:	Very Frequent (multiple times per day for MRT commuters)		
Flow of Events:	1. User inputs the MRT station name in the search field 2. System searches and retrieves matching station(s) 3. System retrieves real-time crowd density data from the transport API 4. The system displays the crowd density level with a visual indicator: - Low (Green): Minimal crowding, plenty of space - Medium (Yellow): Moderate crowding, some space available - High (Red): Heavy crowding, limited space 5. The system automatically updates the crowd density every 10 minutes		
Alternative Flows:	-		
Exceptions:	EX1: Storage/ database error		

	EX2: Invalid MRT station name- Display "Station not found" message
Includes:	-
Special Requirements:	-
Assumptions:	• User accounts support these features
Notes and Issues:	Manage storage for multiple favourites

3.2.3.4 Manage user preferences

Use Case ID:	UC15		
Use Case Name:	Manage user preferences		
Created By:	Prakriti Muralidharan	Last Updated By:	Prakriti Muralidharan
Date Created:	02/09/2025	Date Last Updated:	03/09/2025
Actor	Commuter (public transport)		
Description:	Users can set travel preferences (e.g., mode of transport).		
Preconditions:	• User is logged in		
Postconditions:	• Preferences stored		
Priority:	Medium		
Frequency of Use:	Occasional (when updating the settings)		
Flow of Events:	1. User opens preferences 2. User selects preferred travel options 3. System saves those preferences 4. If the user favourites a transport screen (e.g. if user favourites the bus stop name → derived from the bus stop code), they are added under Favourited Transport screen 5. If the user favourites the carpark, they are added under Bookmarked Carparks screen		
Alternative Flows:	AS-F1: 1. User resets preferences to default (if they want to)		
Exceptions:	EX1: Preferences are not saved due to a system error		
Includes:	-		
Special Requirements:	Preferences should persist across all devices		
Assumptions:	-		

Notes and Issues:	Personalisation enhances the adoption of our app

3.2.4. Functional Requirement #4: Display of Error Messages

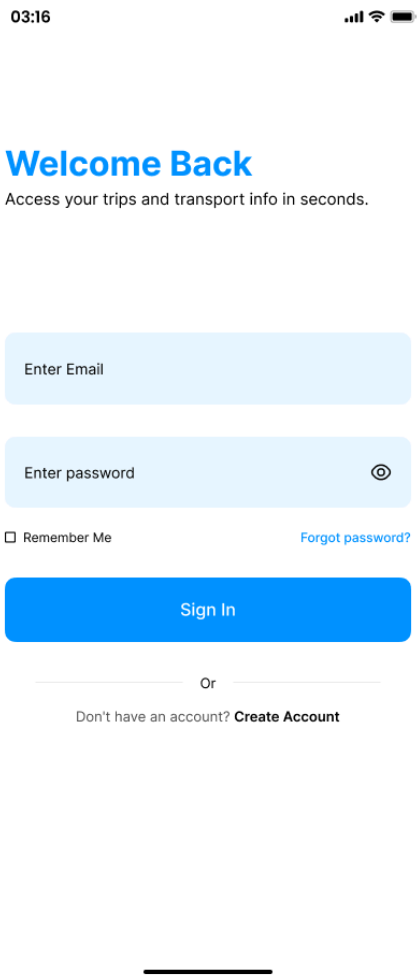
3.2.4.1 Display Error Message

Use Case ID:	UC16		
Use Case Name:	Display Error Message		
Created By:	Toh Kok Soon	Last Updated By:	Toh Kok Soon
Date Created:	01/09/2025	Date Last Updated:	01/09/2025
Actor	User		
Description:	Show error text		
Preconditions:	● User has no account		
Postconditions:	● Account is created and saved in the database		
Priority:	Medium		
Frequency of Use:	Low		
Flow of Events:	1. Display a Pop-up with an error message		
Alternative Flows:	AF-S1: 1. If a designated area for an error message is found, the error message will be rendered on the page itself, and no pop-up will be shown		
Exceptions:	-		
Includes:	-		
Special Requirements:	-		
Assumptions:	-		
Notes and Issues:	An example of a designated area for an error message can be at the bottom of forms, such as login and registration		

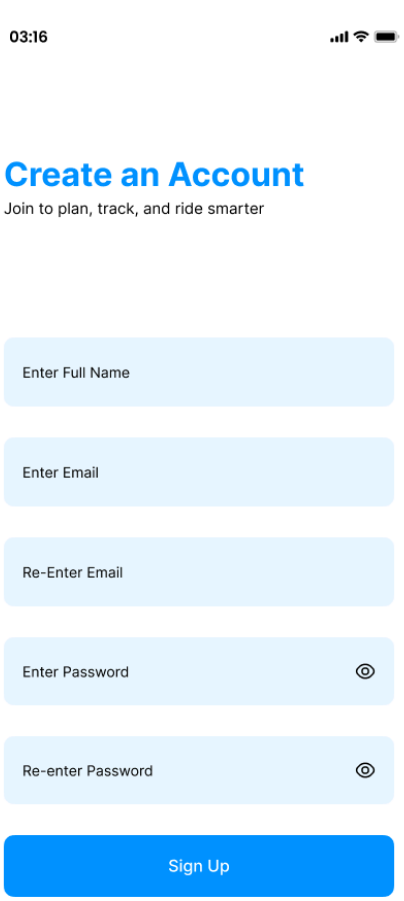
3.3 UI Mock Ups

3.3.1 Account Management

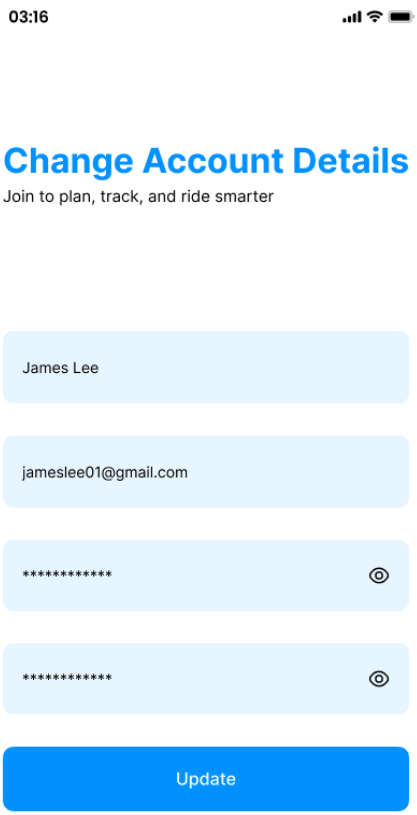
Login Page



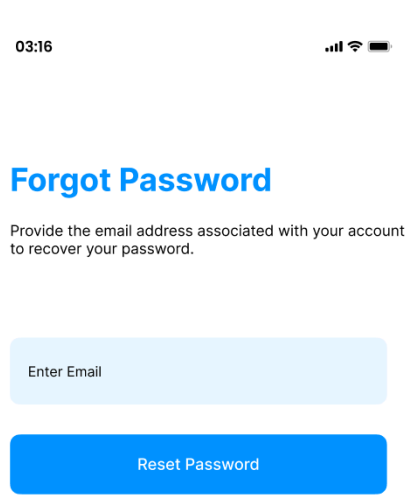
Registration Page

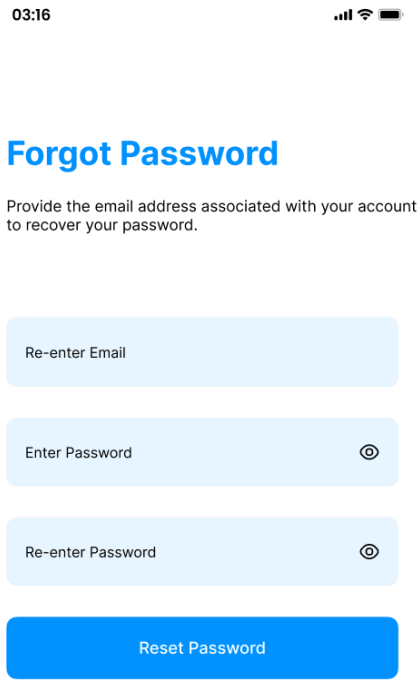


Change Account Details



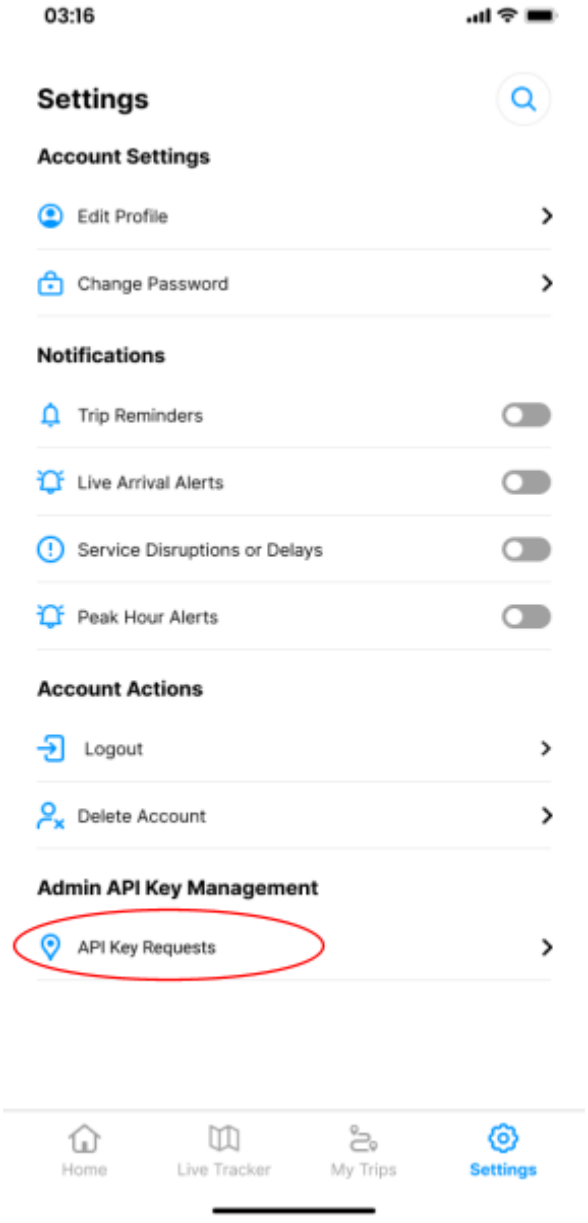
Forgot Password



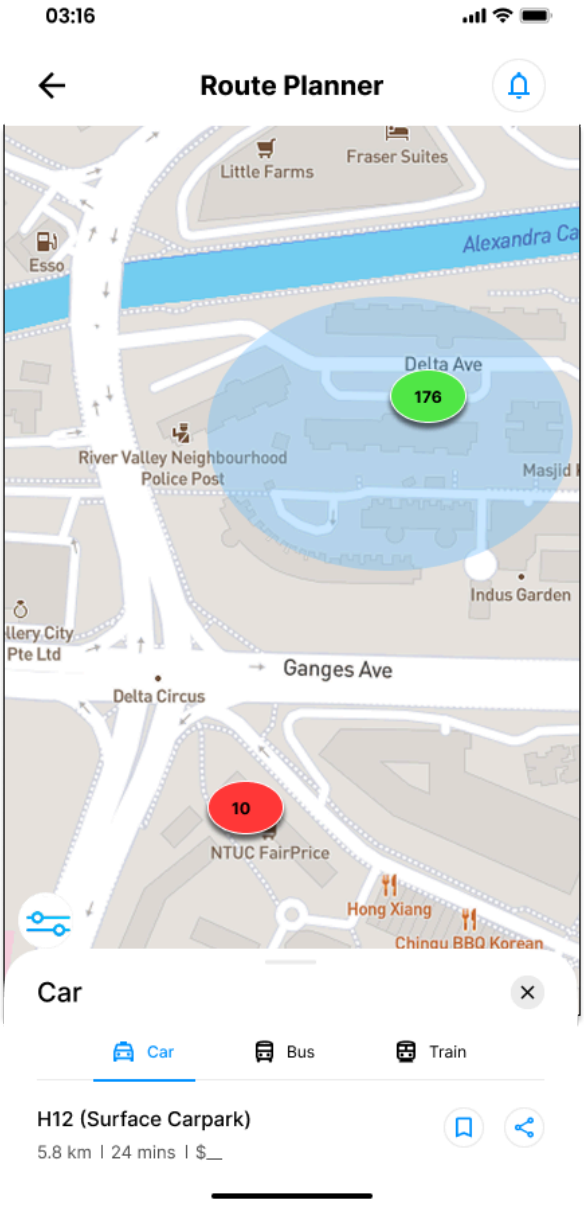


3.3.2 Carpark Availability

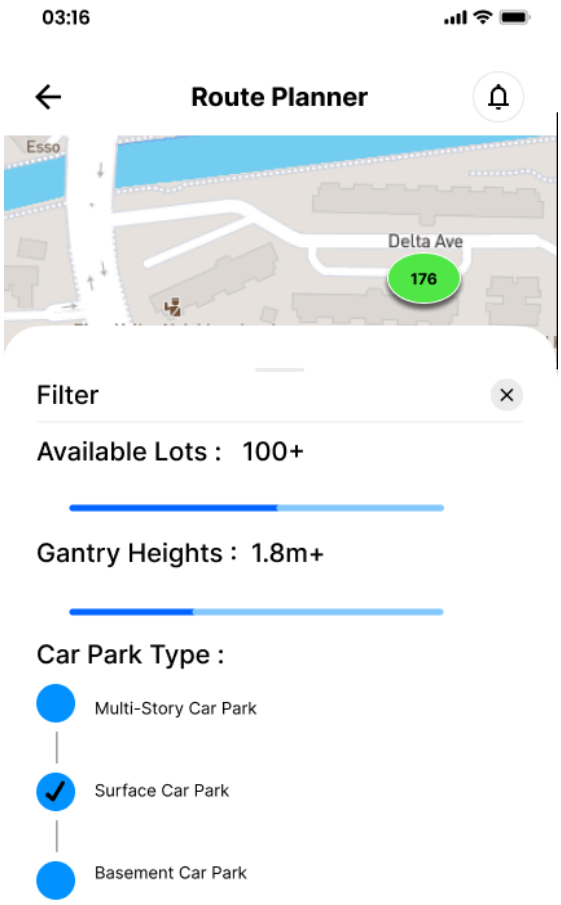
Admin Setting Page - For approval and rejection of API Key



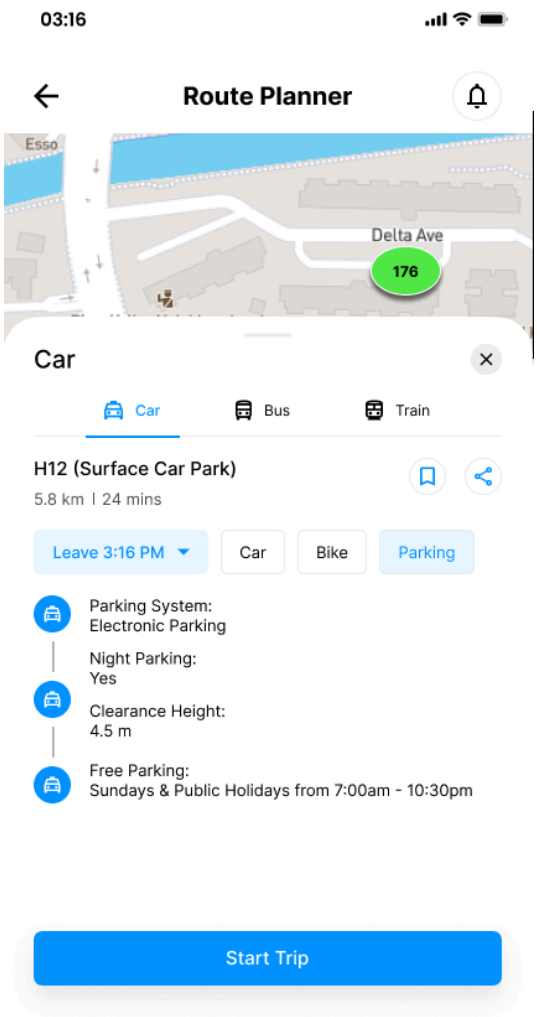
Carpark Map



Carpark Filters

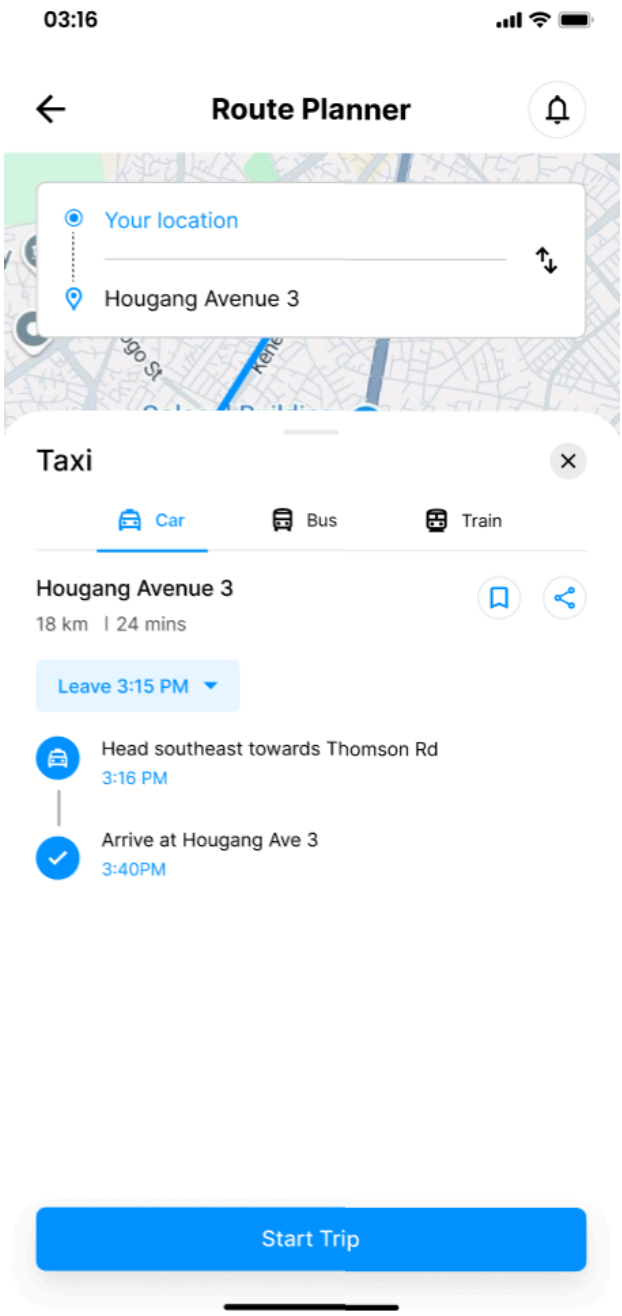


Carpark Details

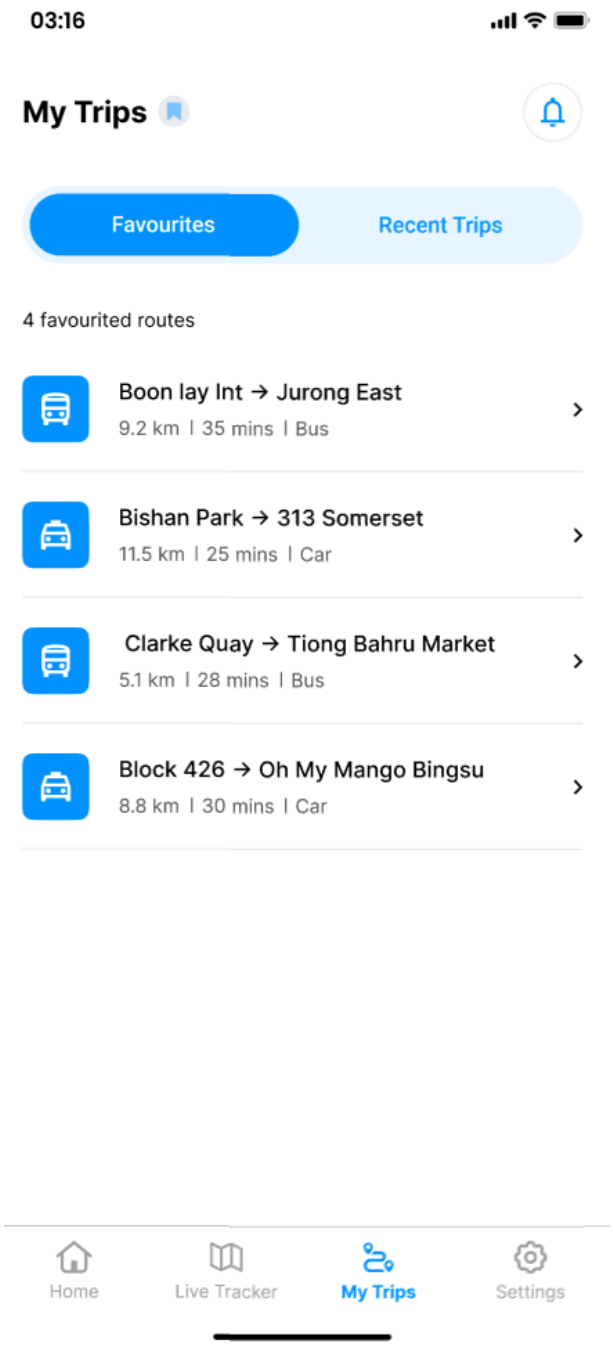


3.3.3 Public Transport Planner

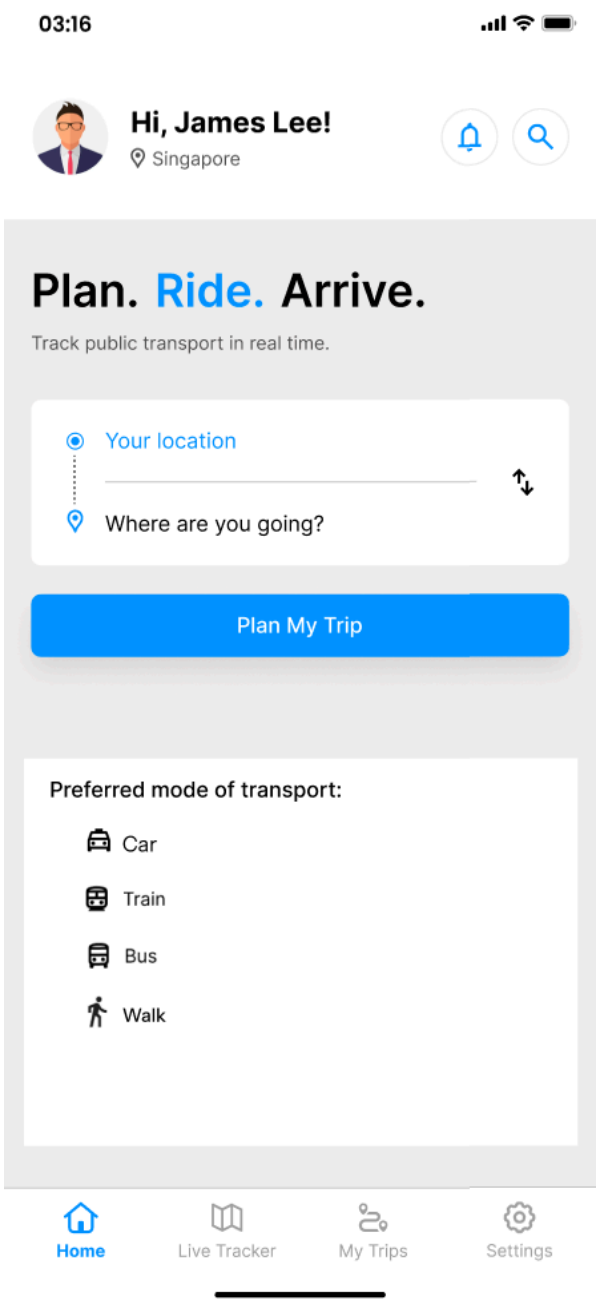
Get real-time arrival information:



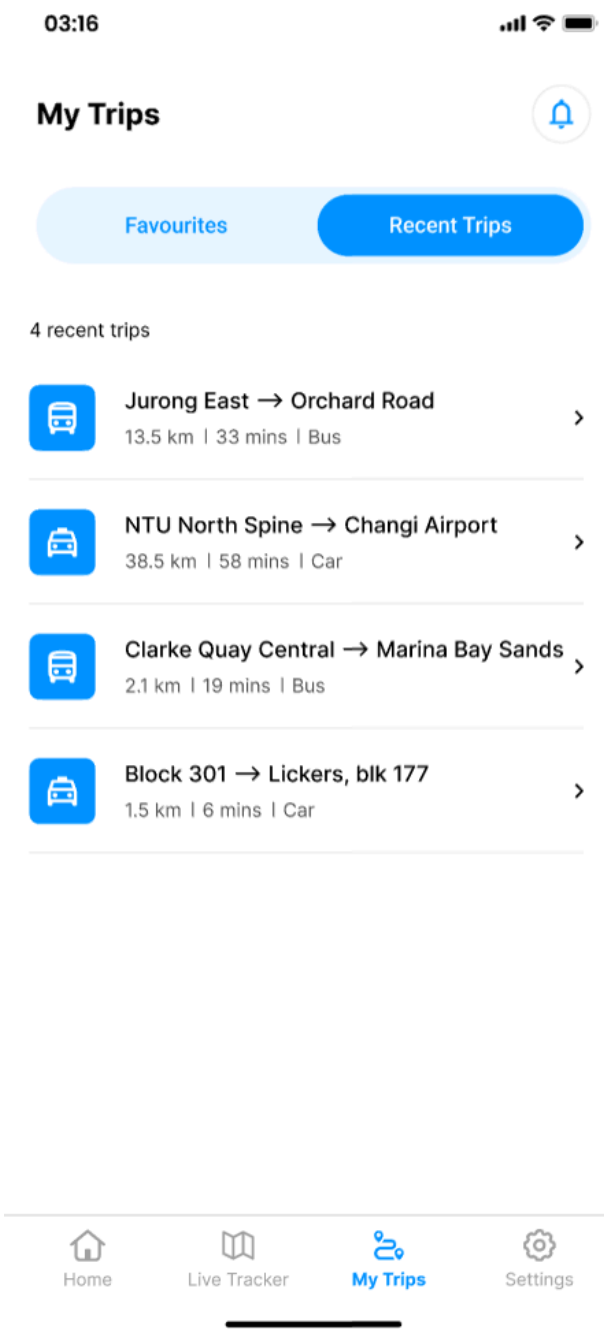
Bookmark favourite routes:



Manage user preferences:

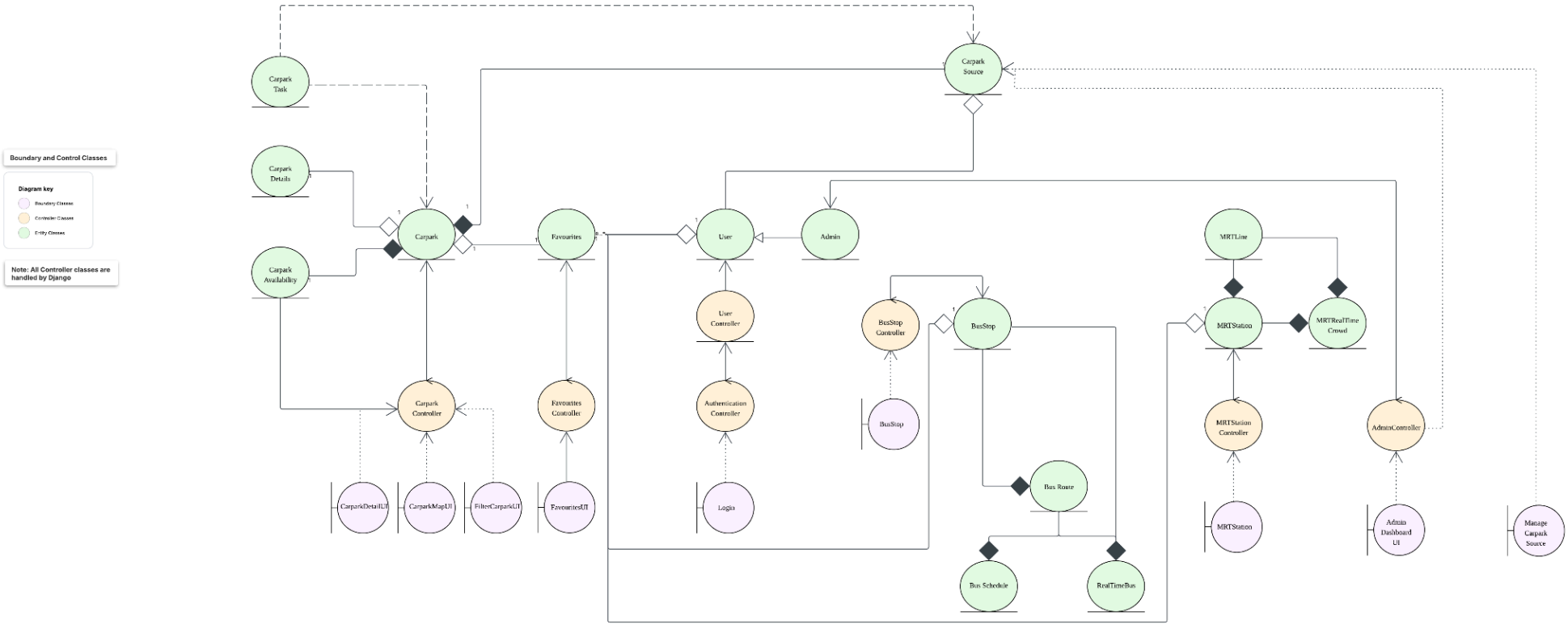


View historical data :



4. Key boundary classes and control classes

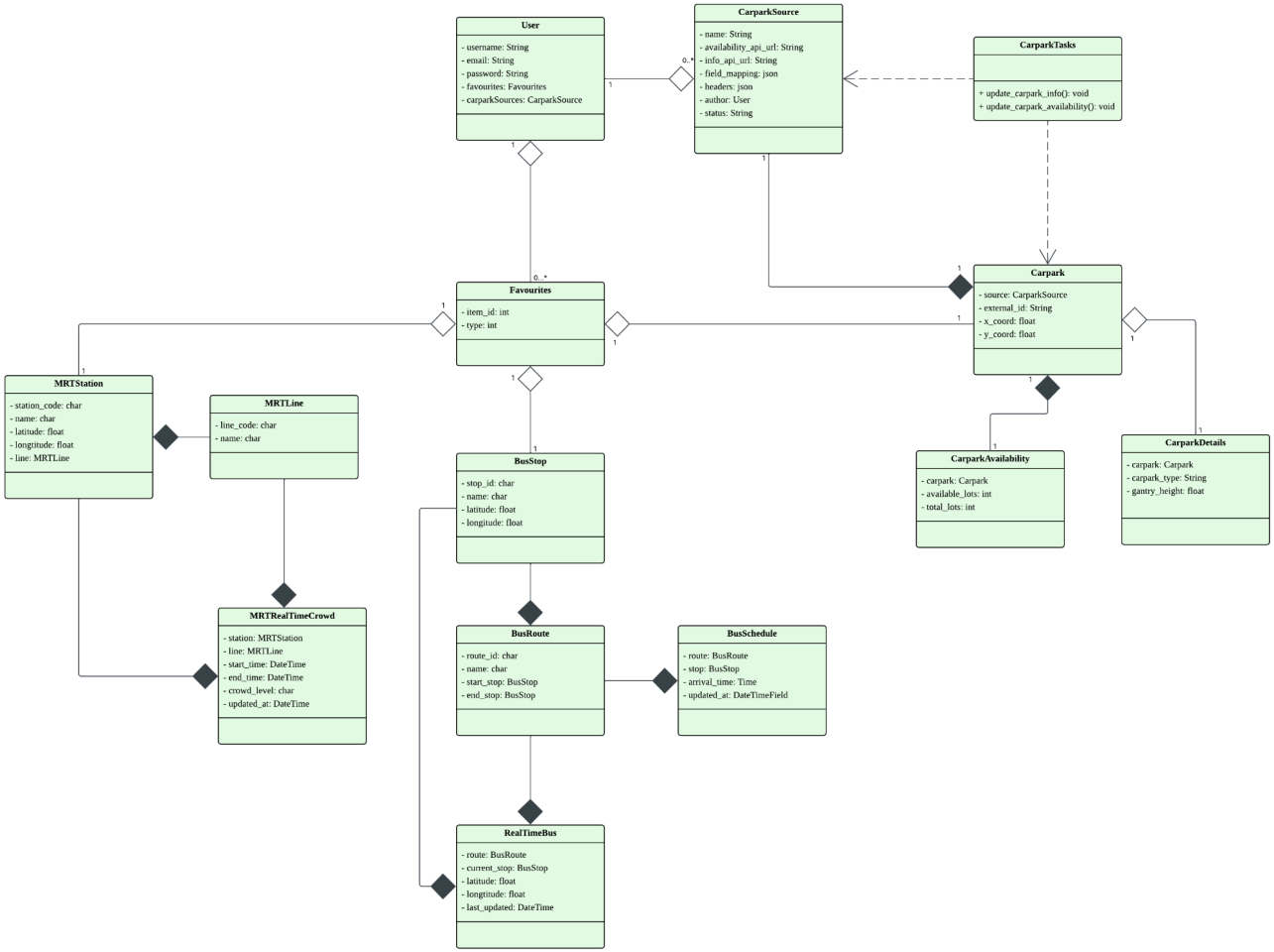
(For a clearer view of the image, please refer to the PNG attached in the GitHub repository.)



All Controller Logic Handled by Django

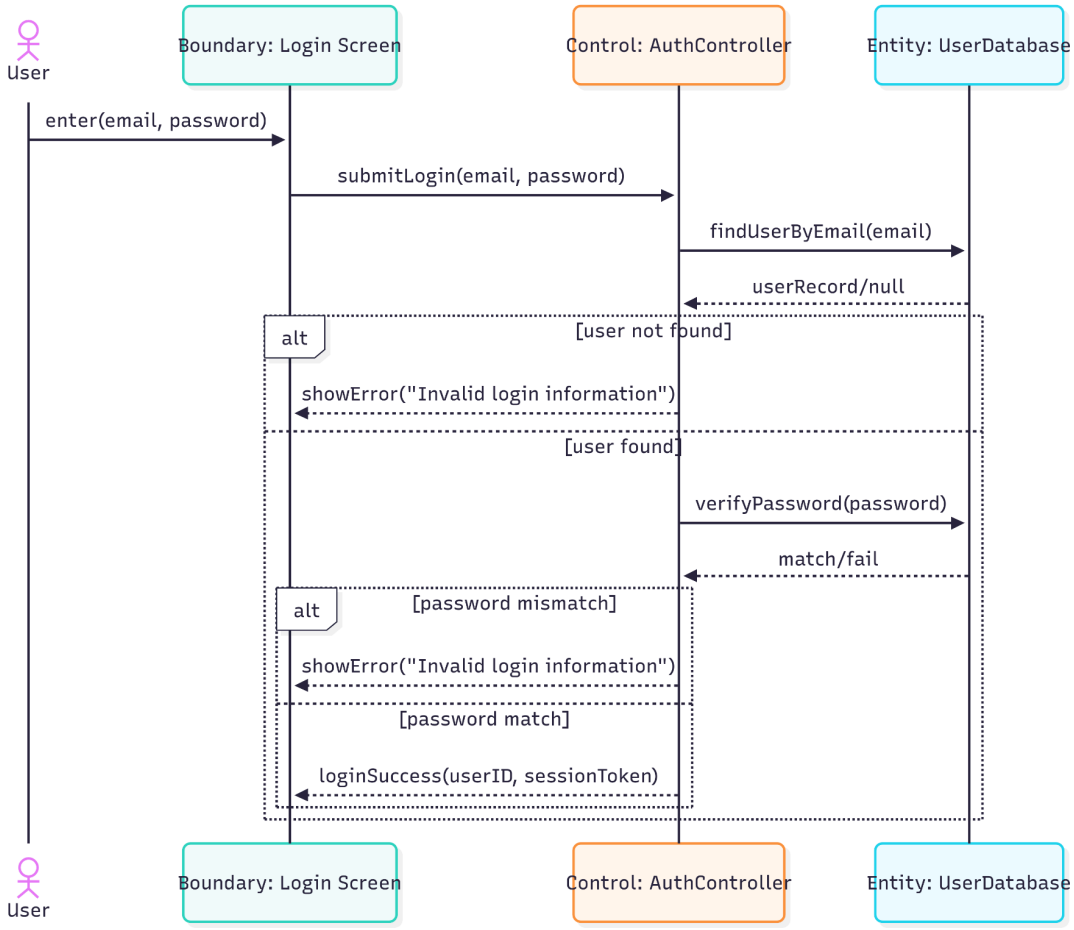
5. Class Diagram

(For a clearer view of the image, please refer to the PNG attached in the GitHub repository.

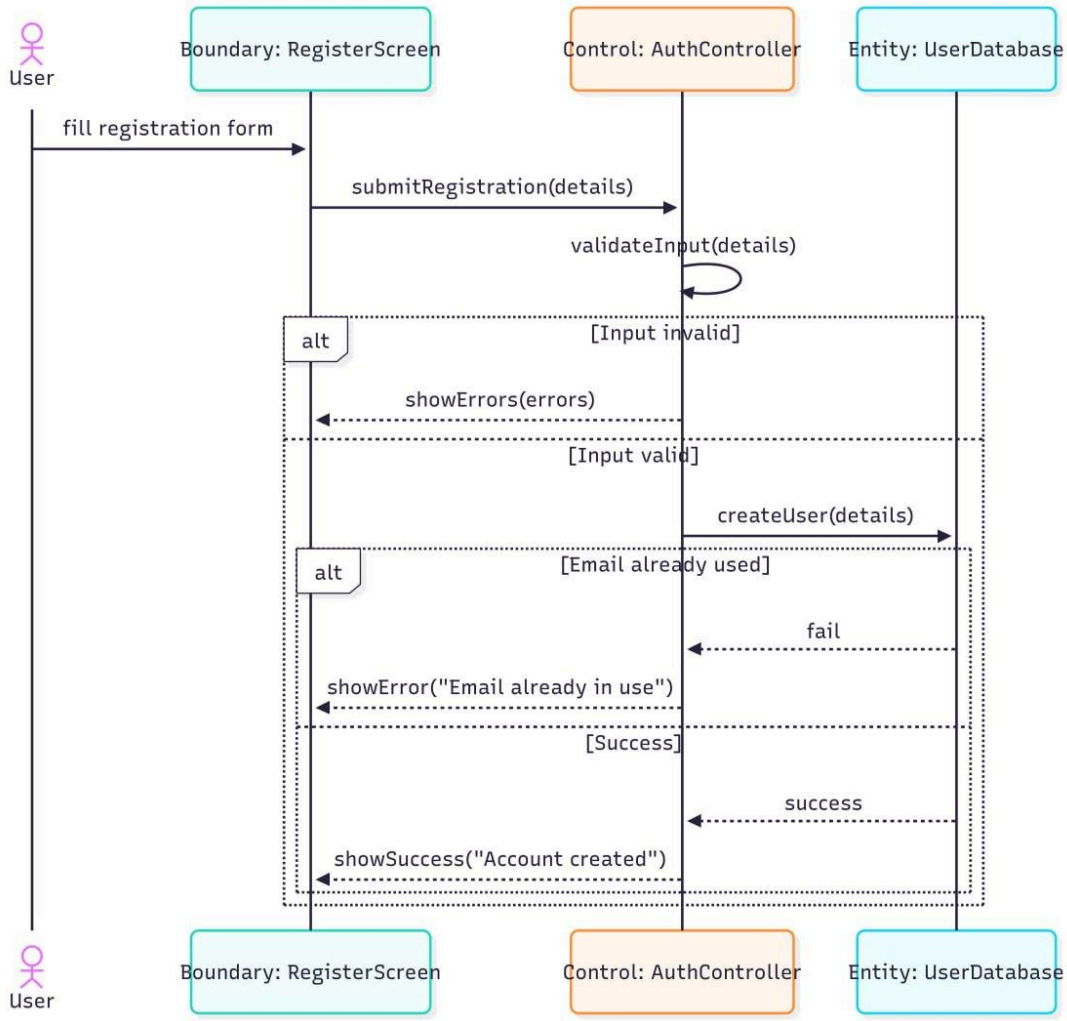


6. Sequence Diagram

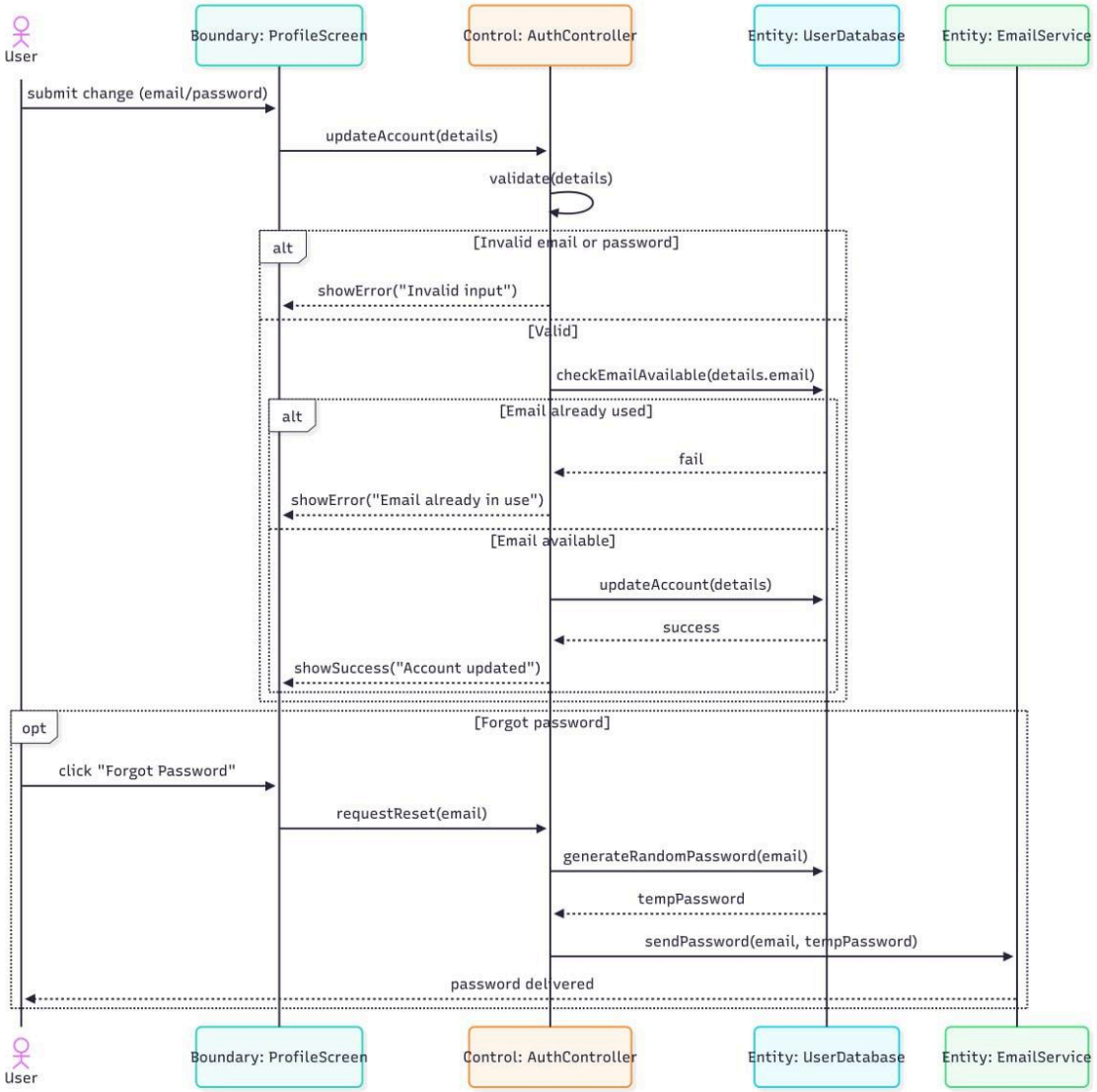
6.1. UC1 - Login



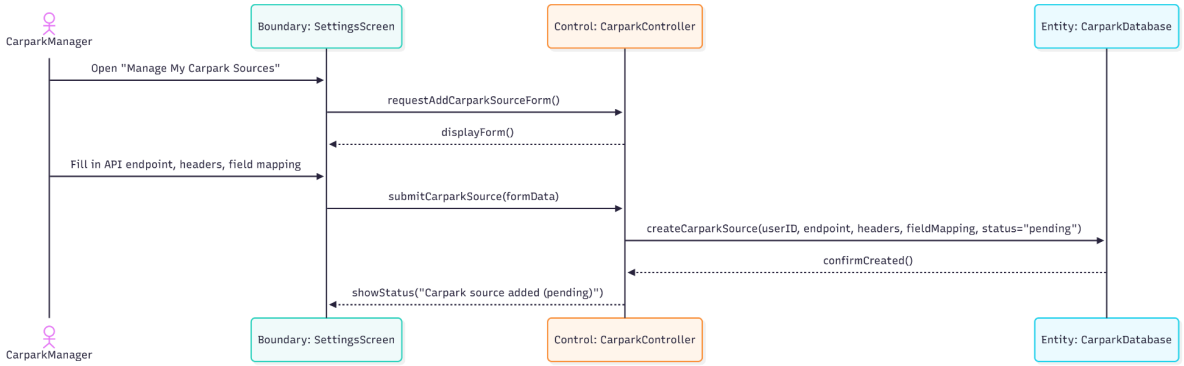
6.2. UC2 - Register



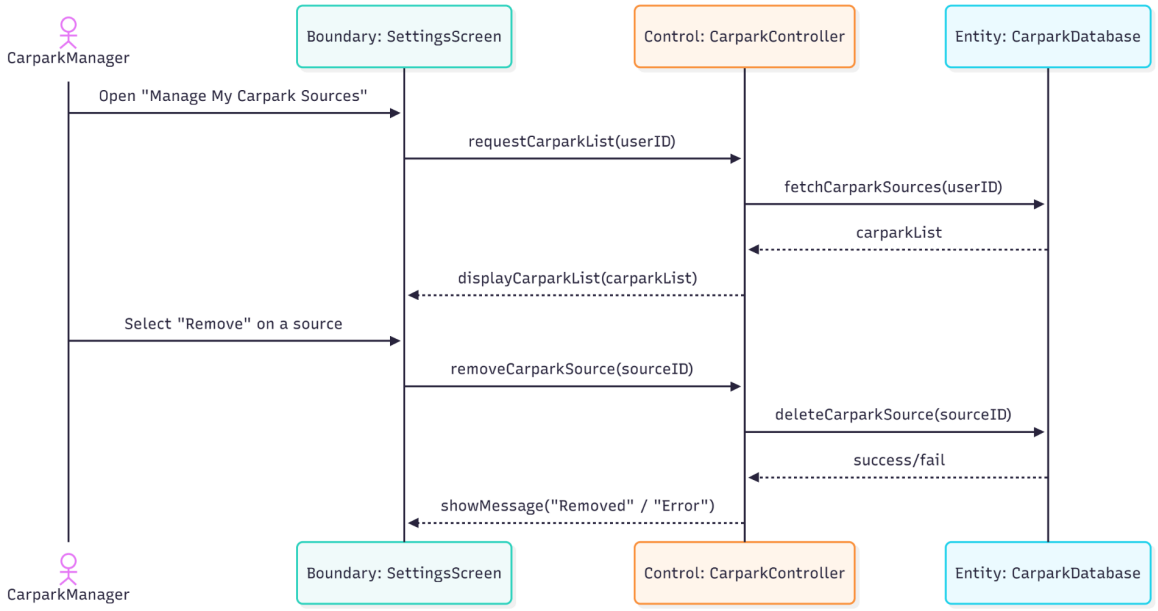
6.3. UC3 - Change Account Detail/ Forget Password



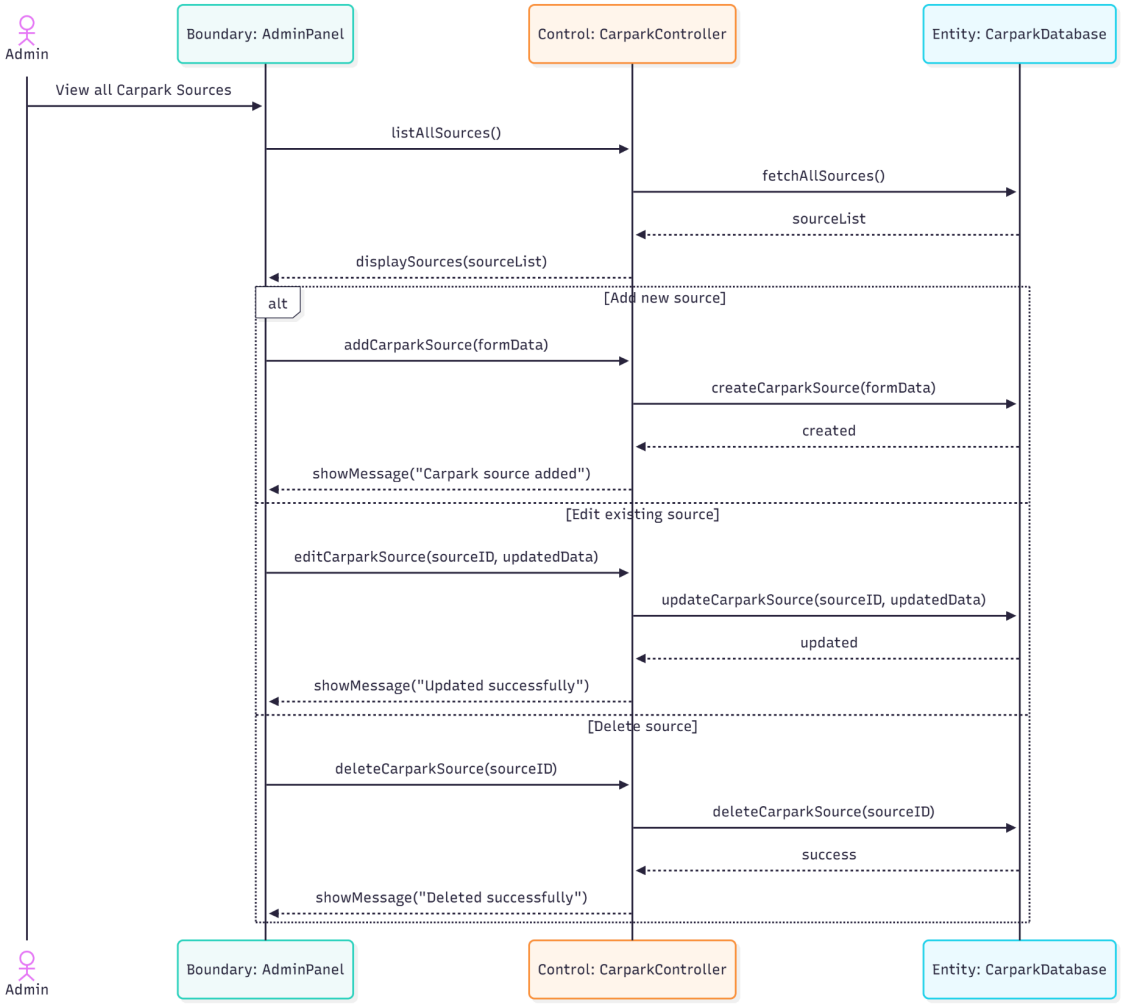
6.4. UC4 - Add Carpark Source



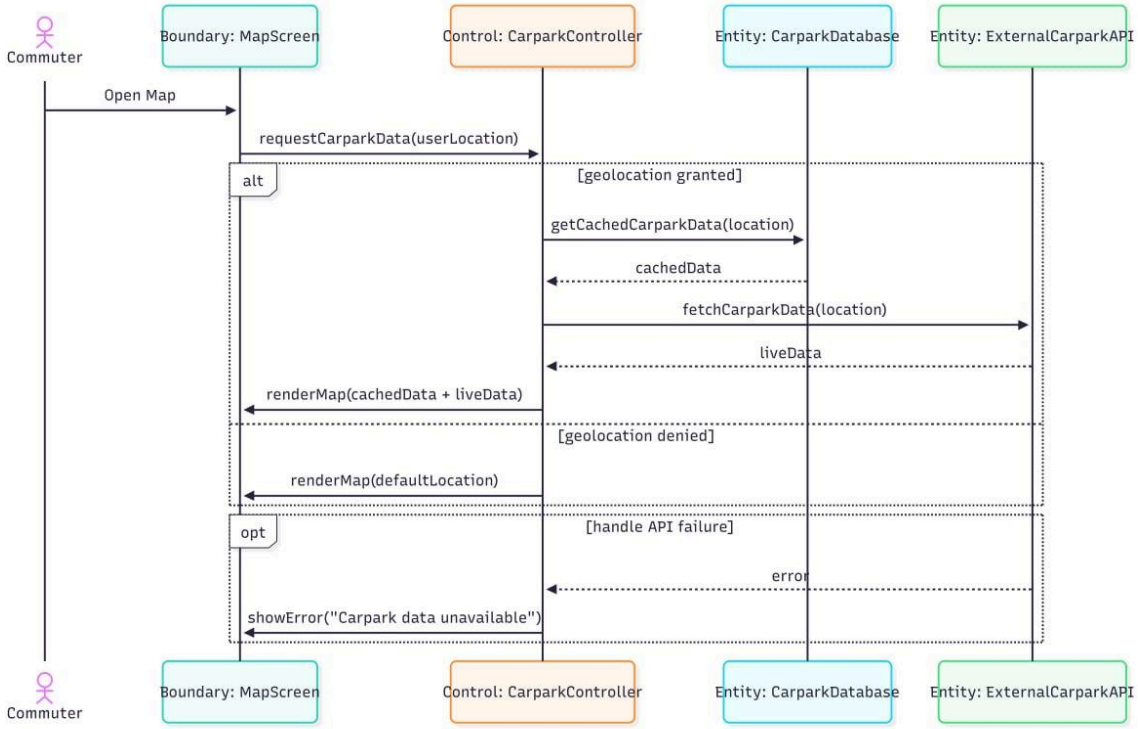
6.5. UC5 - Remove Carpark Source



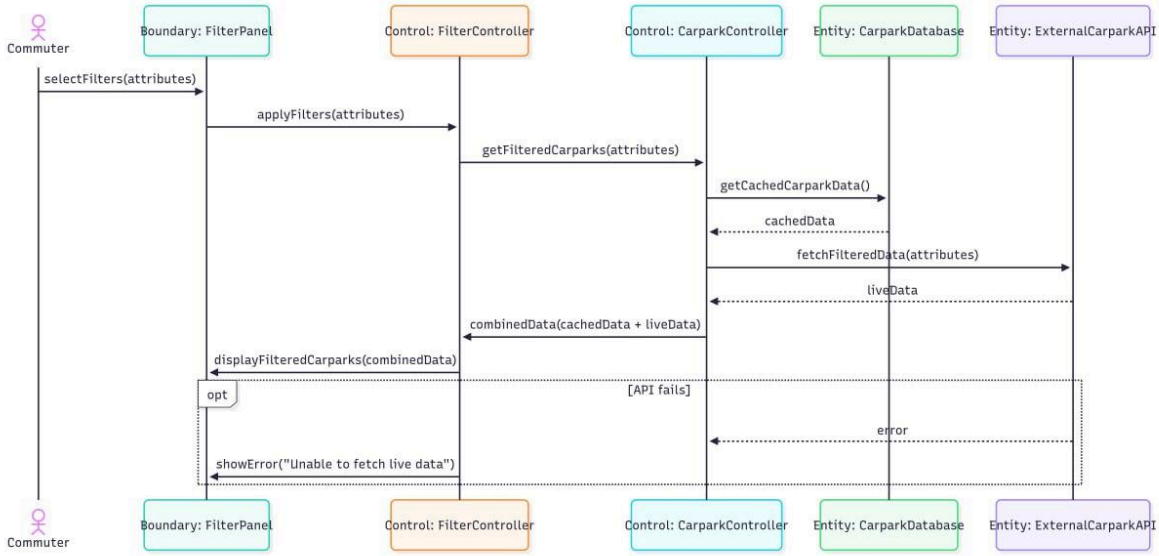
6.6. UC6 - Manage Carpark Source



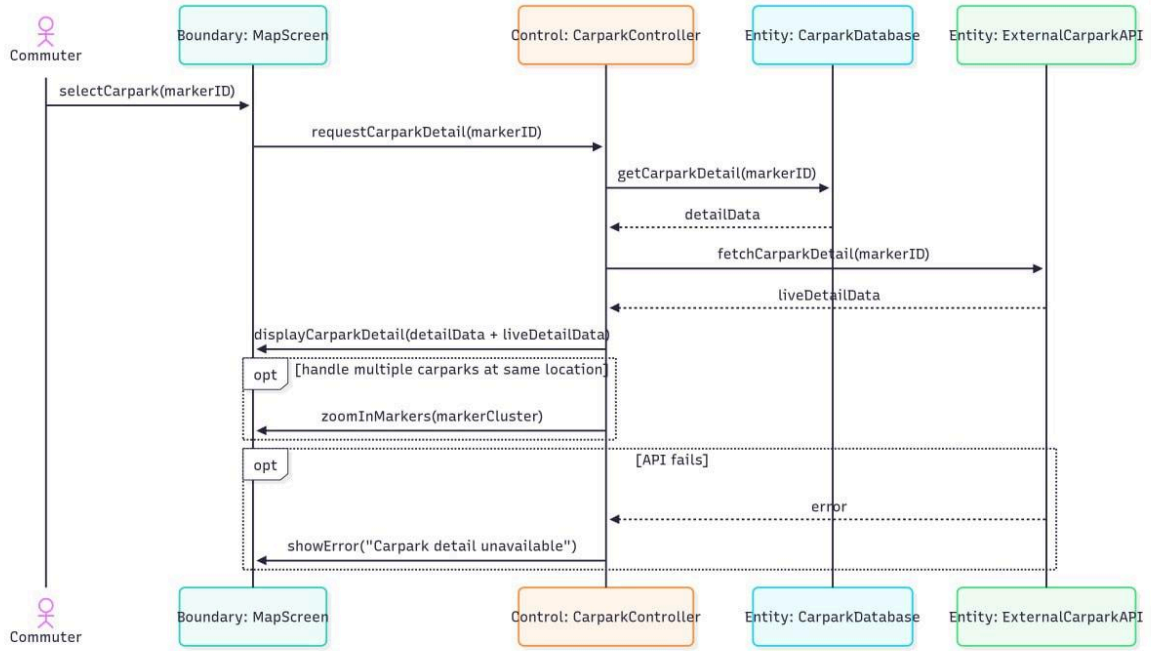
6.7. UC7 - View Carpark Map



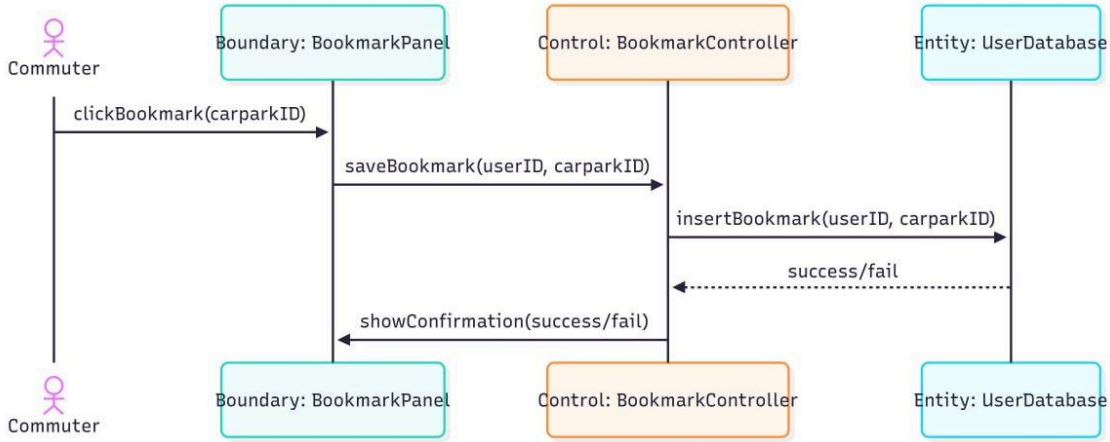
6.8. UC8 - Filter Carpark Map



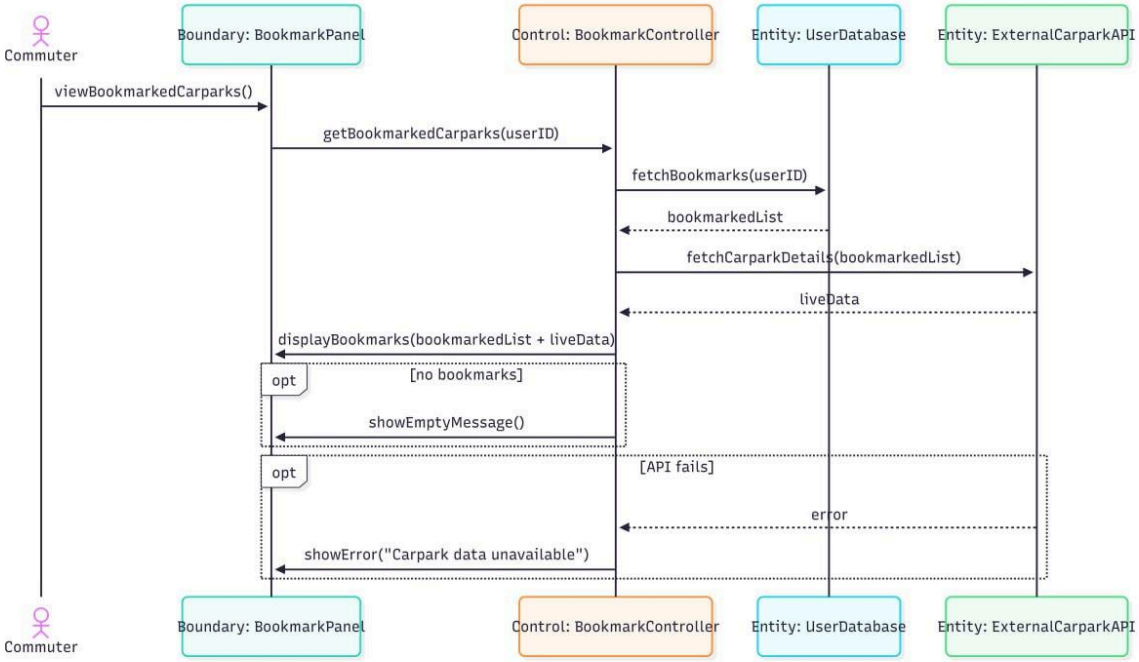
6.9. UC9 - View Carpark Details



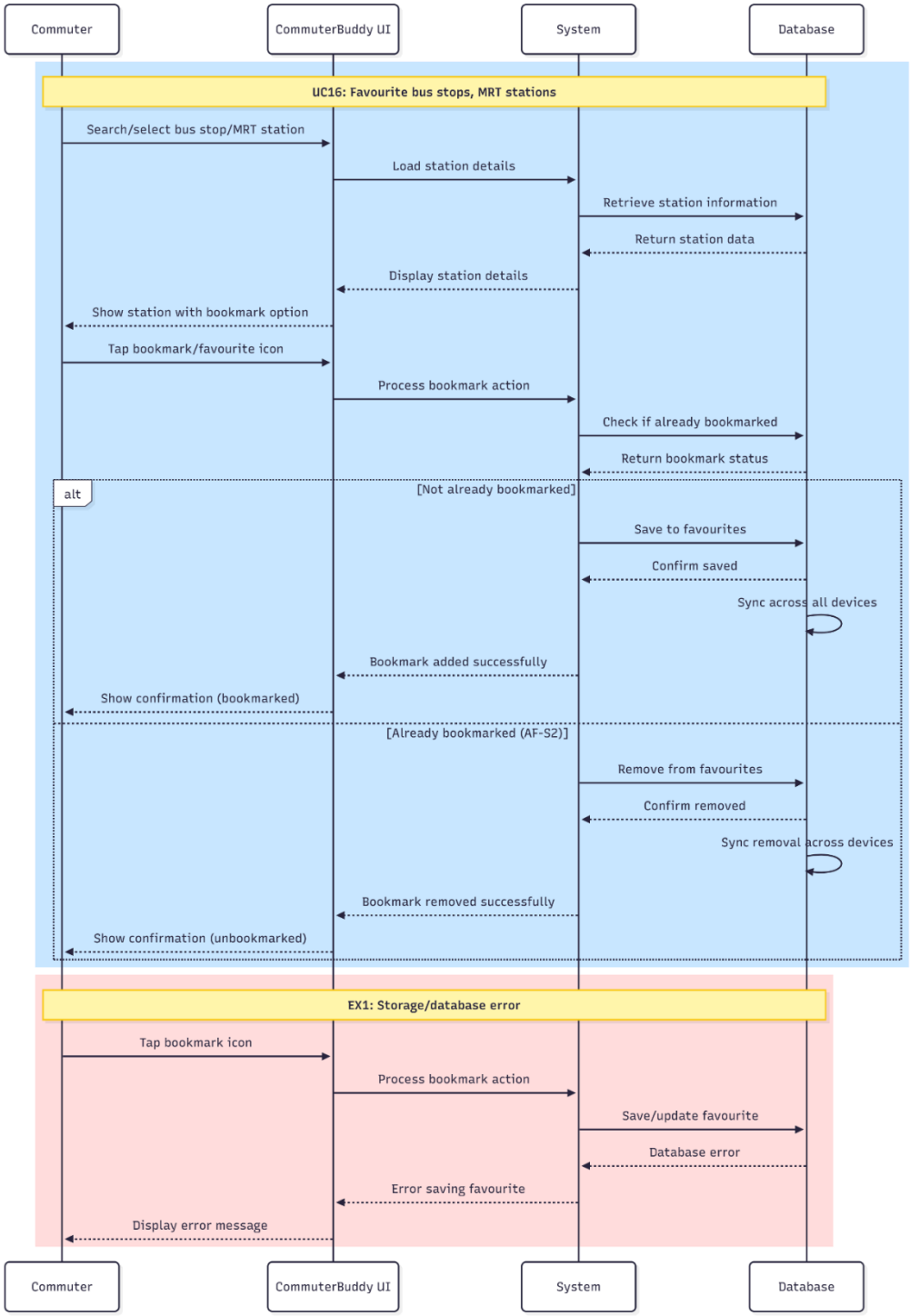
6.10. UC10 - Bookmark Carpark



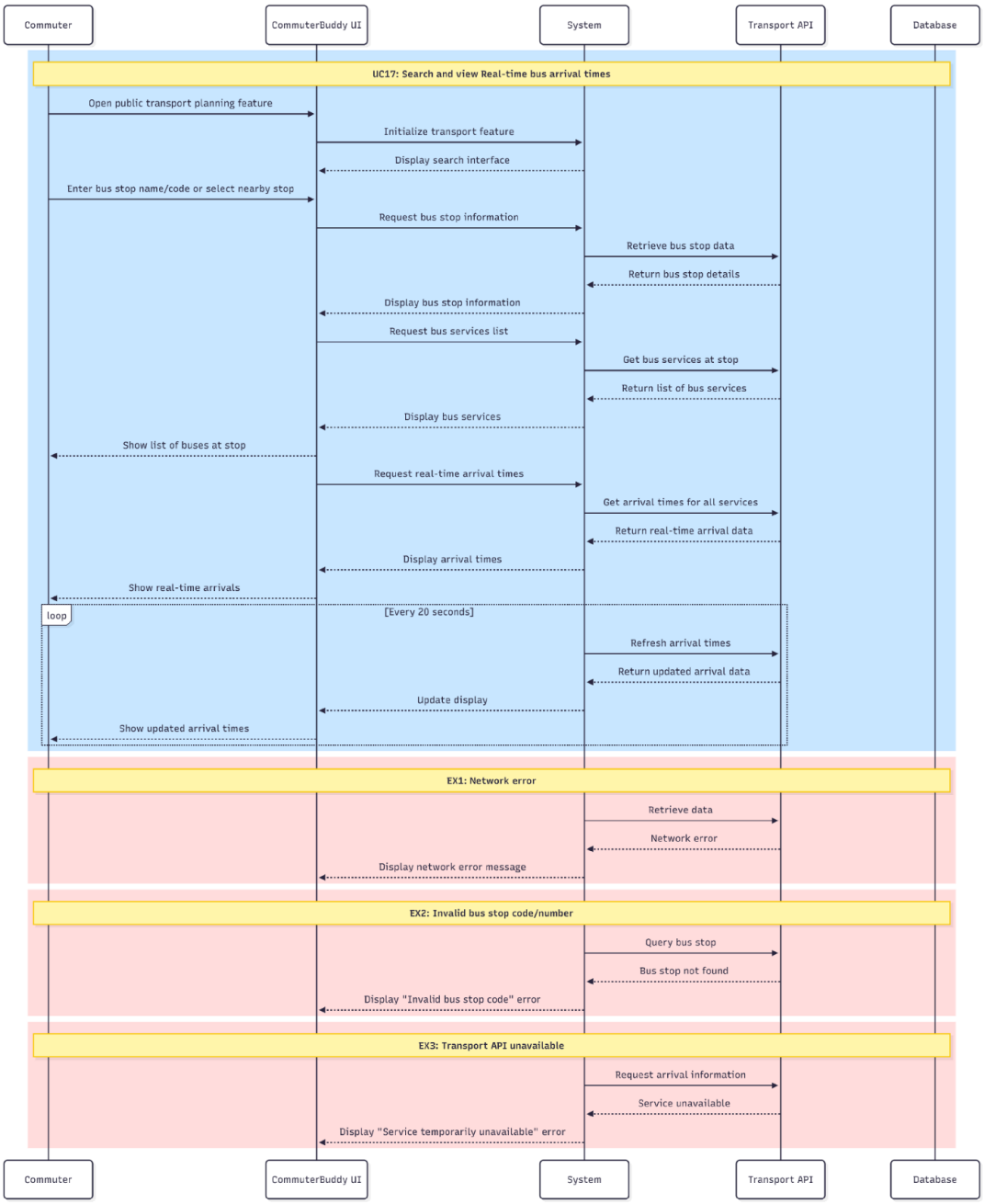
6.11. UC11 - View Bookmark Carpark



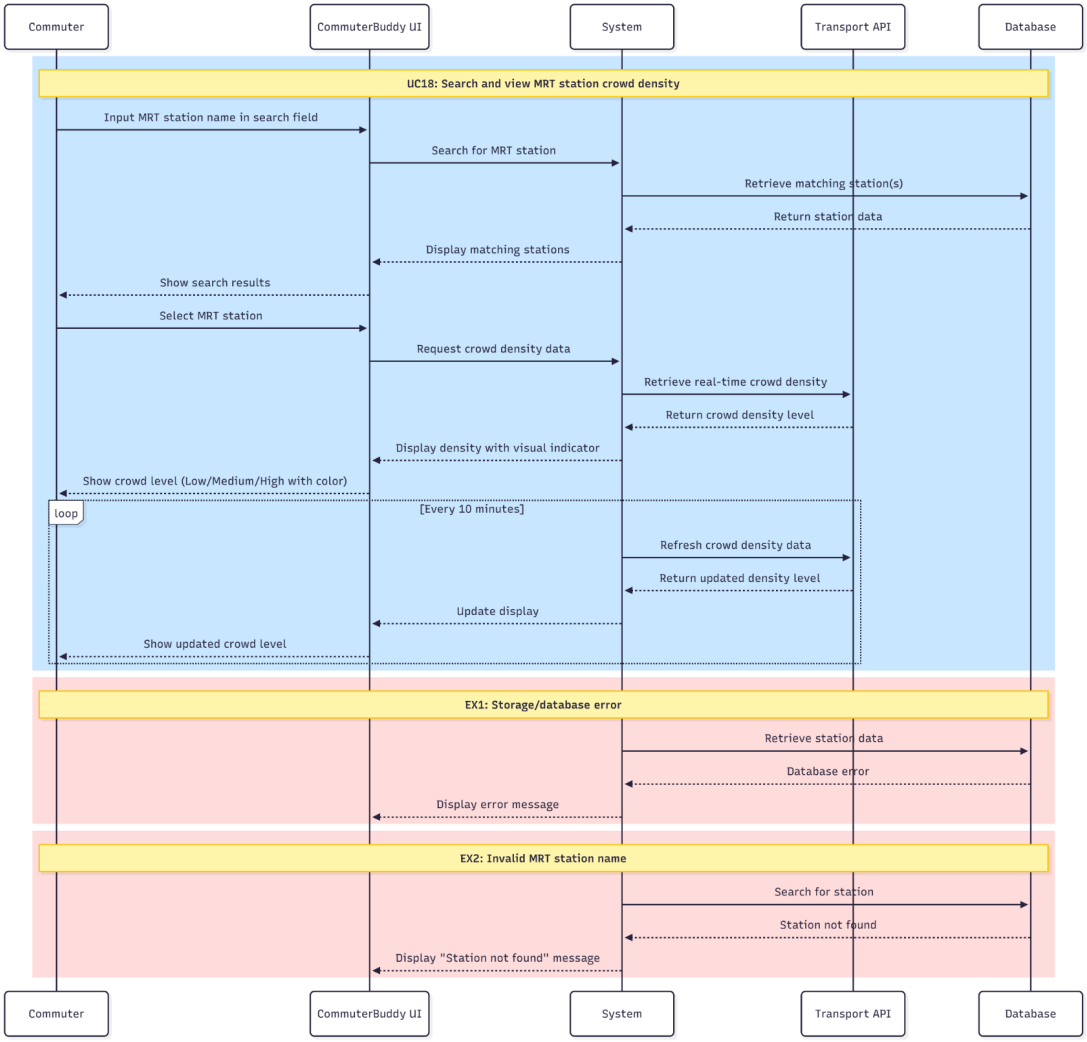
6.12. UC12 - Favourite bus stops, MRT stations



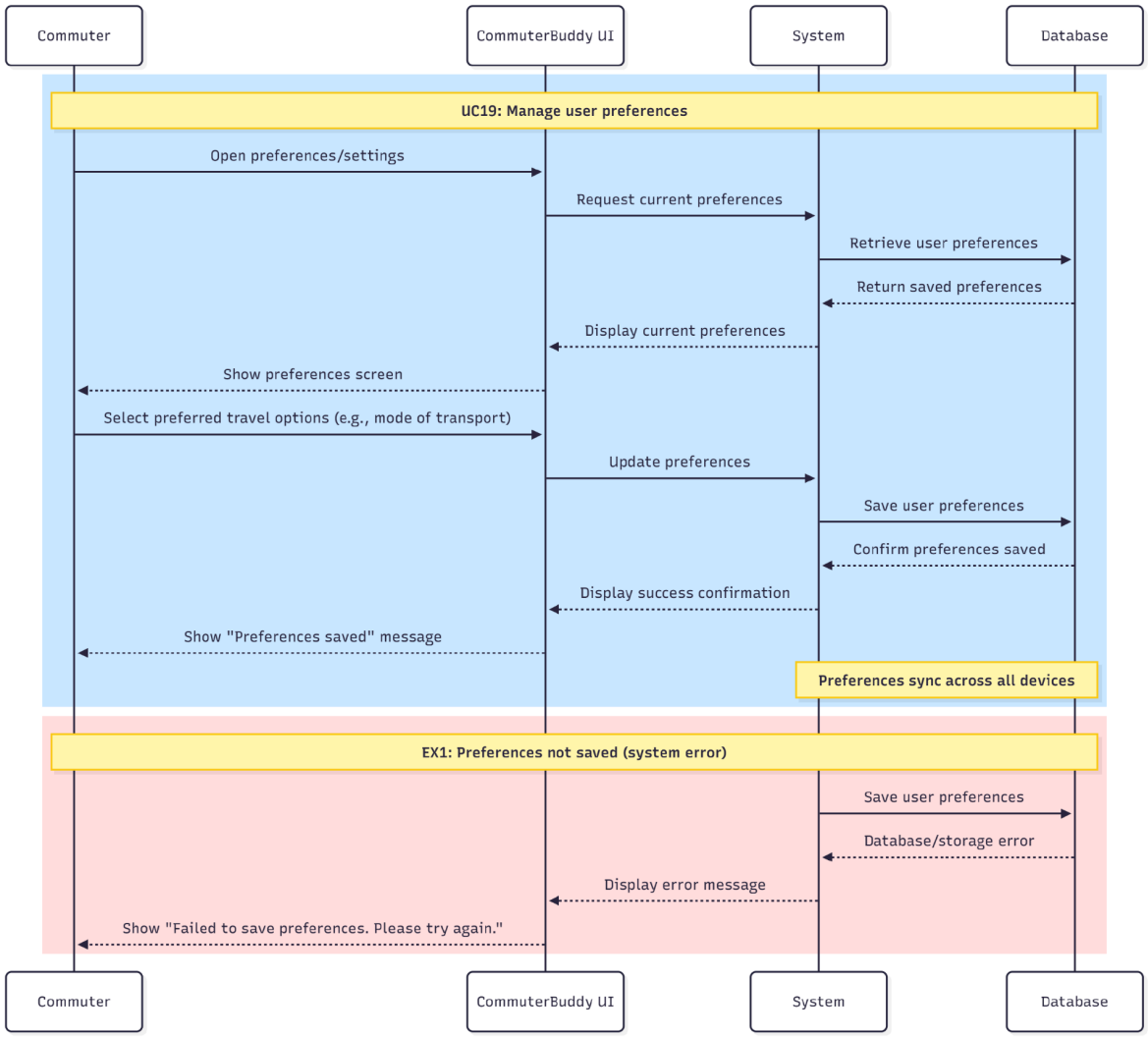
6.13. UC13 - Search and view Real-time bus arrival times



6.14. UC14 - Search and view MRT station crowd density

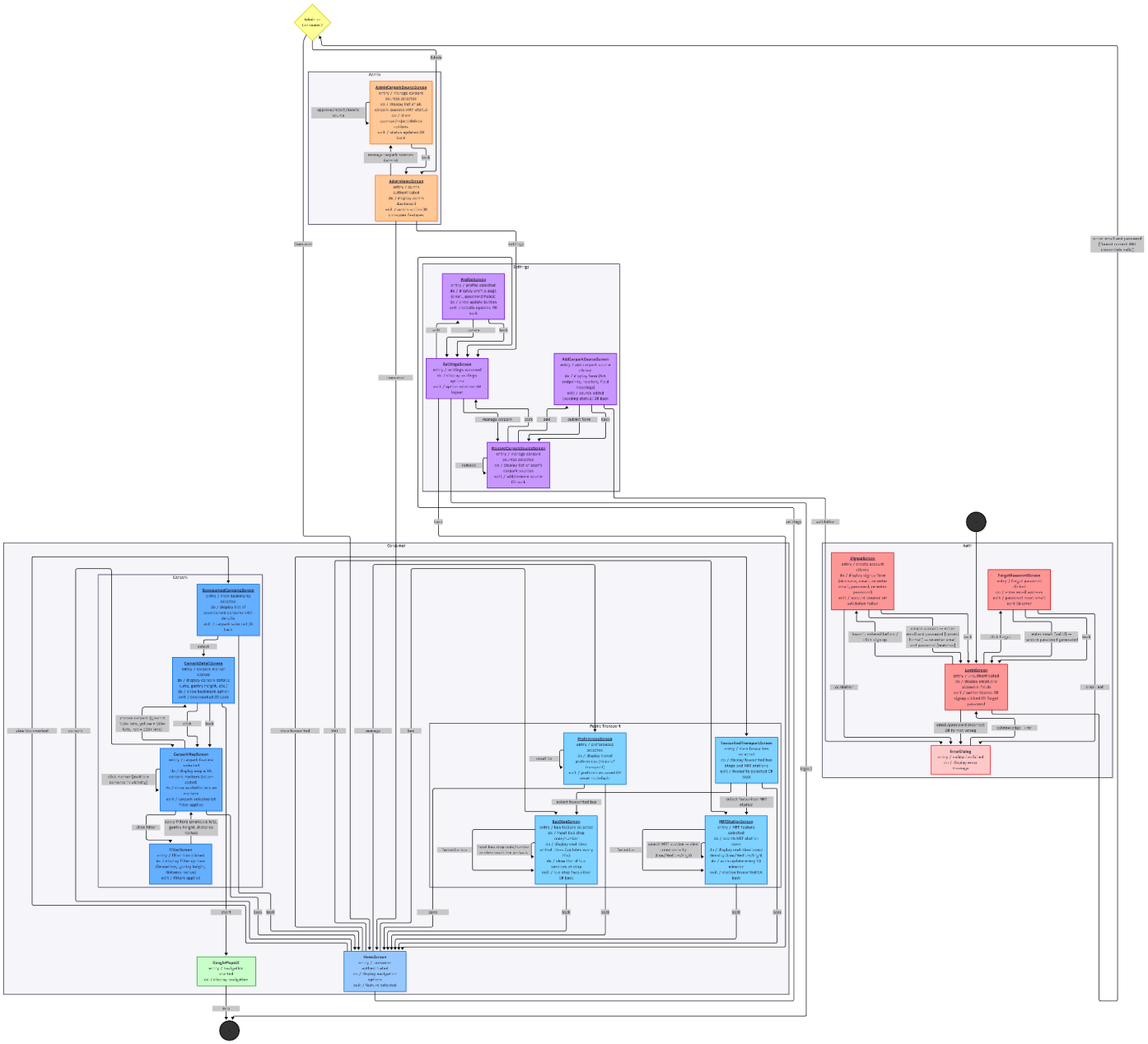


6.15. UC15 - Manage User Preferences



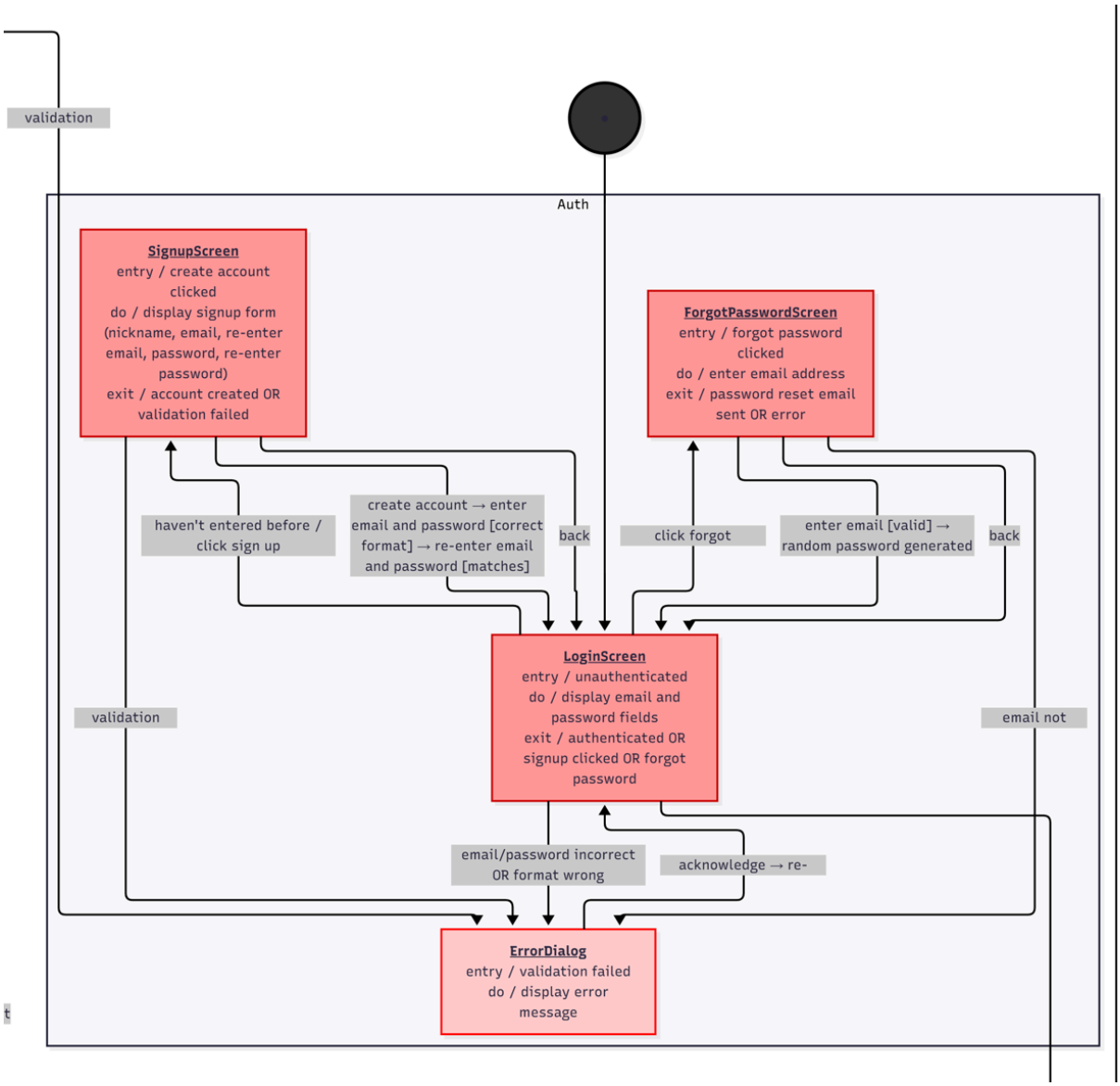
7. Dialog Map

7.1. Overview



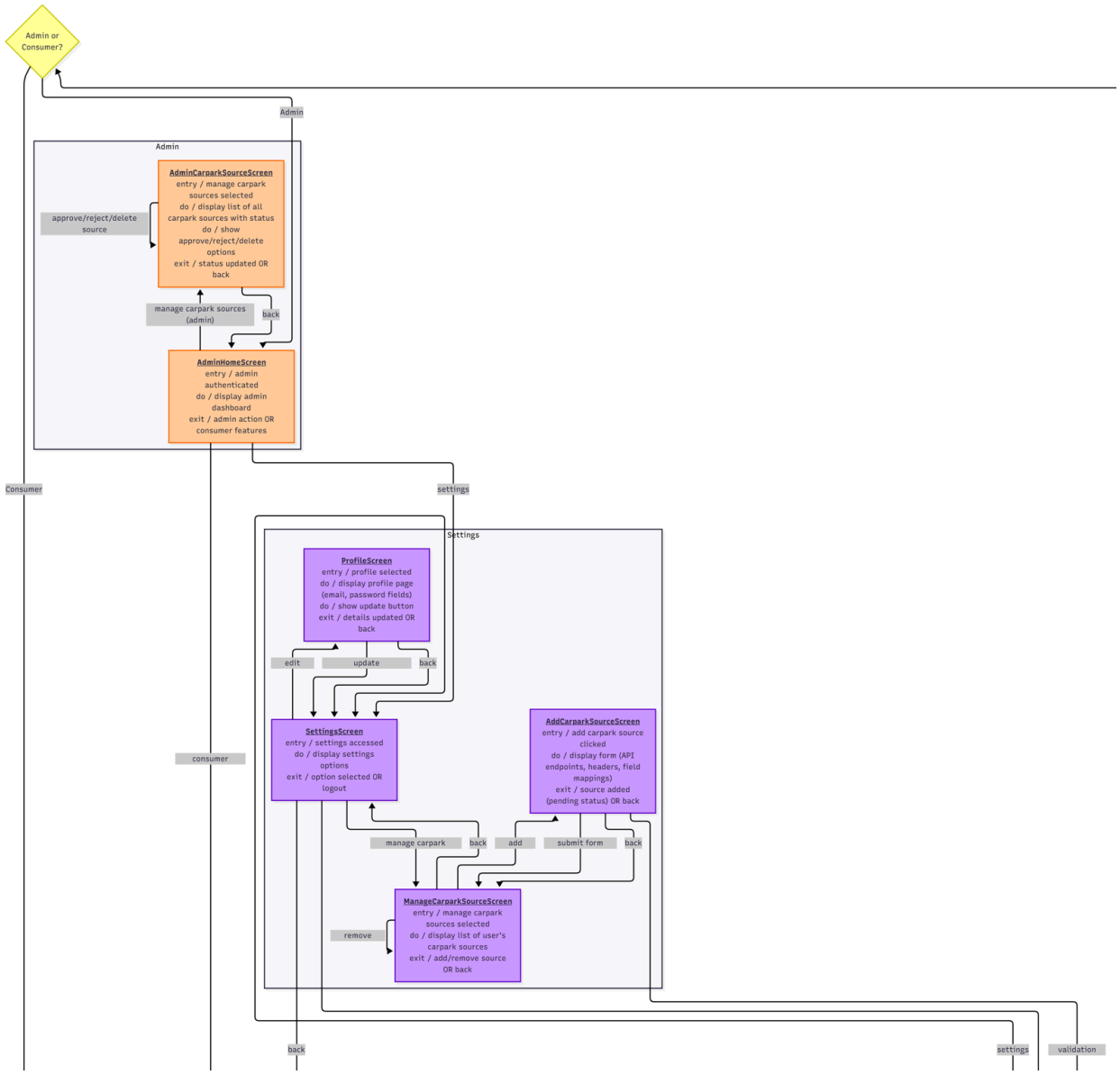
(For a clearer view of the image, please refer to the PNG attached in the GitHub repository.)

7.2. Authentication



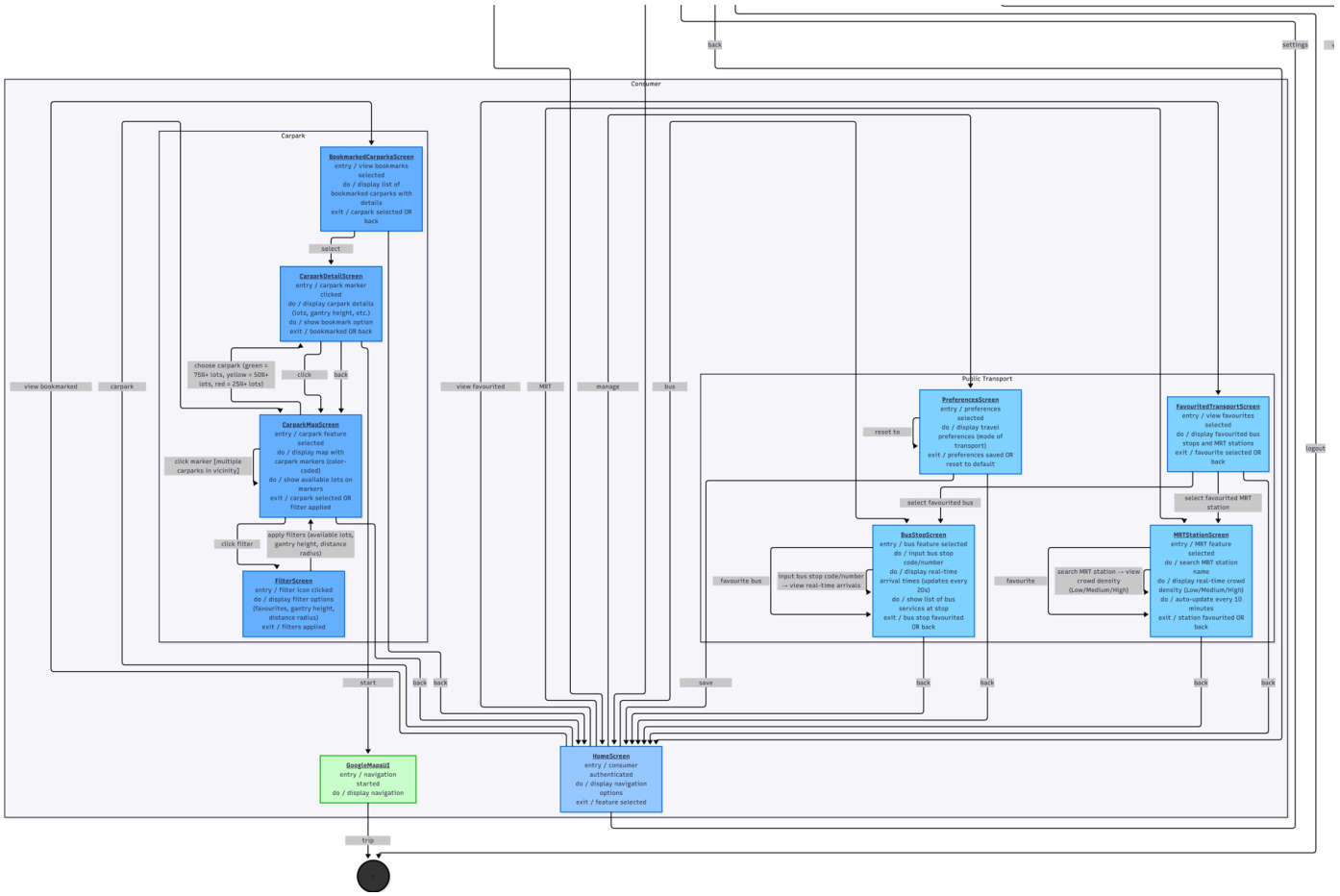
(For a clearer view of the image, please refer to the PNG attached in the GitHub repository.)

7.3. Consumer Settings and Admin Settings



(For a clearer view of the image, please refer to the PNG attached in the GitHub repository.)

7.4. Consumer Features



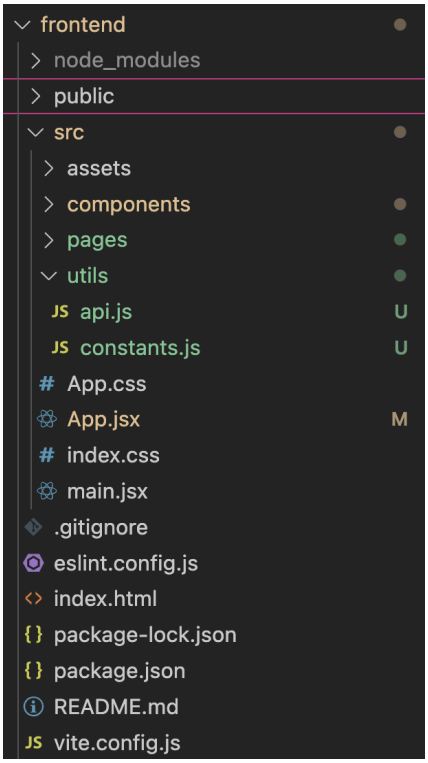
(For a clearer view of the image, please refer to the PNG attached in the GitHub repository.)

8. Application Skeleton

(Please refer to the source code uploaded in the github repository for the full project structure)

8.1 Frontend

The frontend is built using React.js with Vite as the build tool. The structure is organized to promote modularity and ease of collaboration across different components of the application.

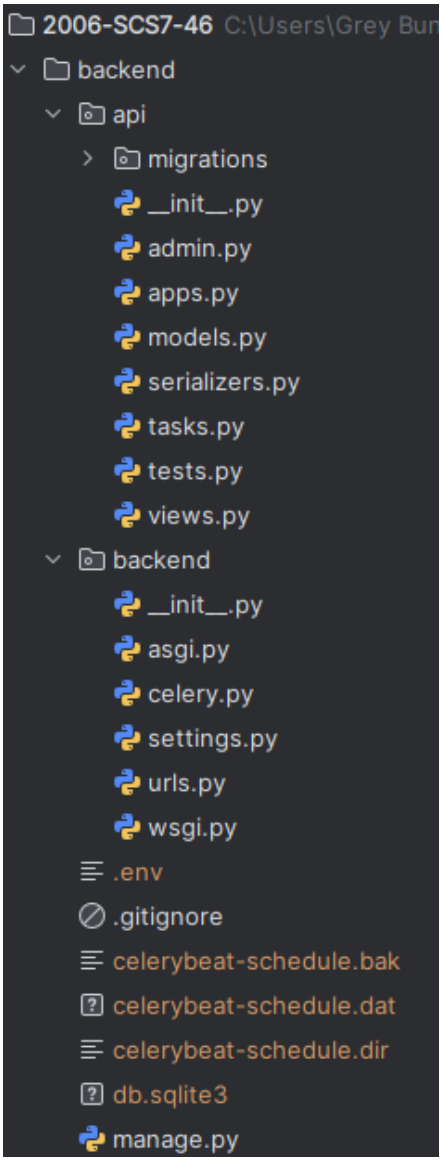


- src/pages/: Contains all the page-level components or screens of the application. Each page represents a major UI view or route in the app.
- src/components/: Includes reusable UI components (buttons, cards, forms, etc.) used across multiple pages to maintain consistency and reduce redundancy.
- api.js: Implement logic for automatically adding headers with authentication token when sending api requests to backend
- constants.js: Defines constants and environment-specific configurations.
- src/assets/: Stores images, icons, and other static resources used throughout the UI.
- App.jsx: Serves as the main entry point of the frontend application. It manages the routing and integration of different pages and components.
- main.jsx: The root file that initializes the React app.
- Other configuration files:
    - vite.config.js: Handles Vite's build and server configurations.

Overall, this structure enhances scalability and readability, making it easier for multiple developers to collaborate and maintain the project efficiently.

8.2 Backend

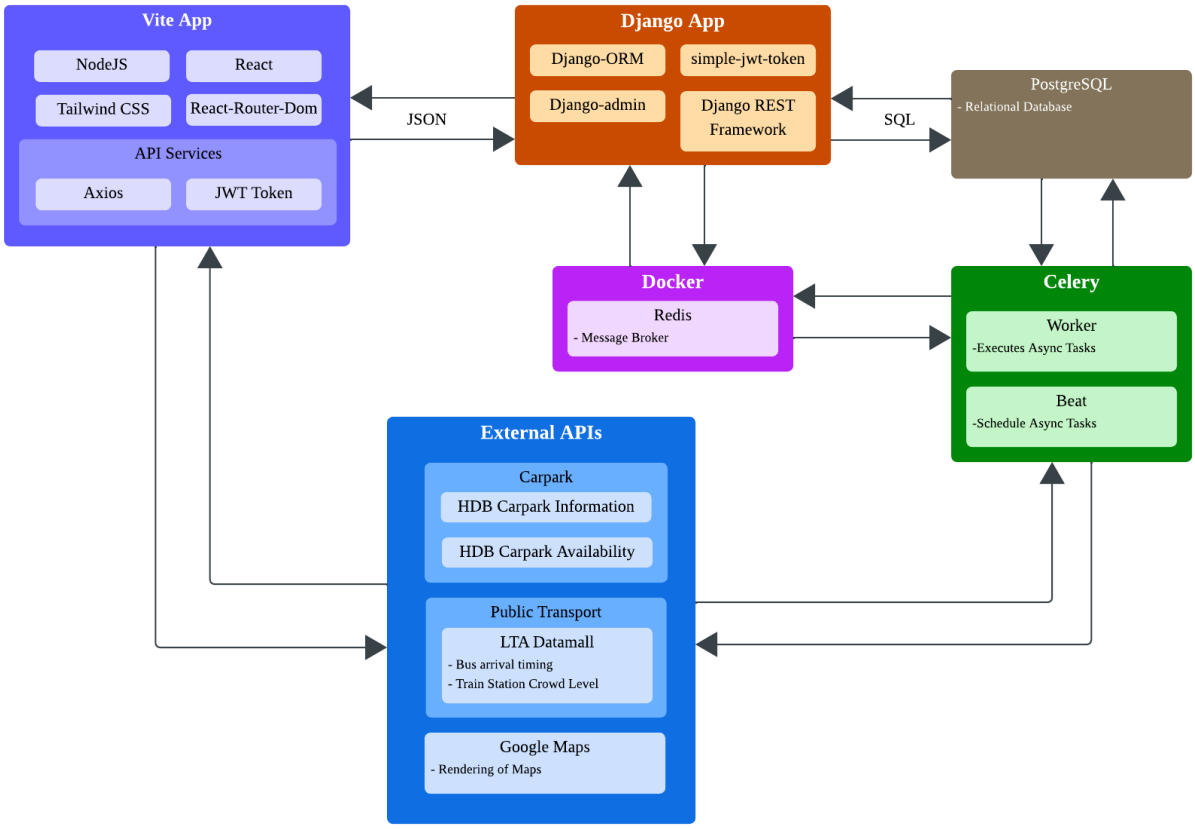
The Django backend handles all database operations. The frontend is able to request for these operations through RESTful API calls. The Django backend also handles the delegation of background tasks such as external API polling to a task queue with Celery.



- backend/: This folder contains the Django project configuration files.
    - settings.py: Central configuration file for installed apps, databases, and middleware.
    - urls.py: Defines all the URL routes for the backend api.
    - celery.py: Defines celery specific settings
- api/: Contains the main application logic and APIs for the backend.
    - models.py: Defines the database schema objects.
    - serializers.py: Handles conversion of JSON to django model objects
    - admin.py: Registers models to the Django admin interface for easy management.

    - migrations/: Tracks database schema changes to keep models and the database in sync.
- db.sqlite3: Local SQLite database file used to store application data for development environment (Will change to postgreSQL).
- manage.py: The command-line utility for administrative tasks such as running the server, applying migrations, and managing apps.

9. System Architecture Diagram



(For a clearer view of the image, please refer to the PNG attached in the GitHub repository.)

10. Test Cases and Testing

10.1 Black Box Testing

10.1.1 CreateUserView (Controller Class)

Django Framework controller logic is handled by view classes. The CreateUserView handles the logic behind the creation of a User. On success, this would lead to the creation of the User object in the database.

10.1.2 Equivalence Class and Boundary Value Testing

CreateUserView inherits from the Django Generic CreateAPIView which is used for a create-only endpoints. It accepts a JSON object through the POST request from the frontend. In our case, the method we are testing is the create method.

create method

Valid Equivalence Class: Username, Email Address and Password with valid format
Invalid Equivalence Class: Username, Email Address and Password with invalid format or already exist

10.1.3 Test Cases and Results

No.	Test Input	Expected Output	Actual Output	Pass?
1	Successful Login email="testuser@gmail.com" username = "testuser" password="password1234@"	"Registration successful!"	"Registration successful!"	Yes
2	Empty Username email="testuser@gmail.com" username = "" password="password1234@"	"Please Fill in This Field"	"Please Fill in This Field"	Yes

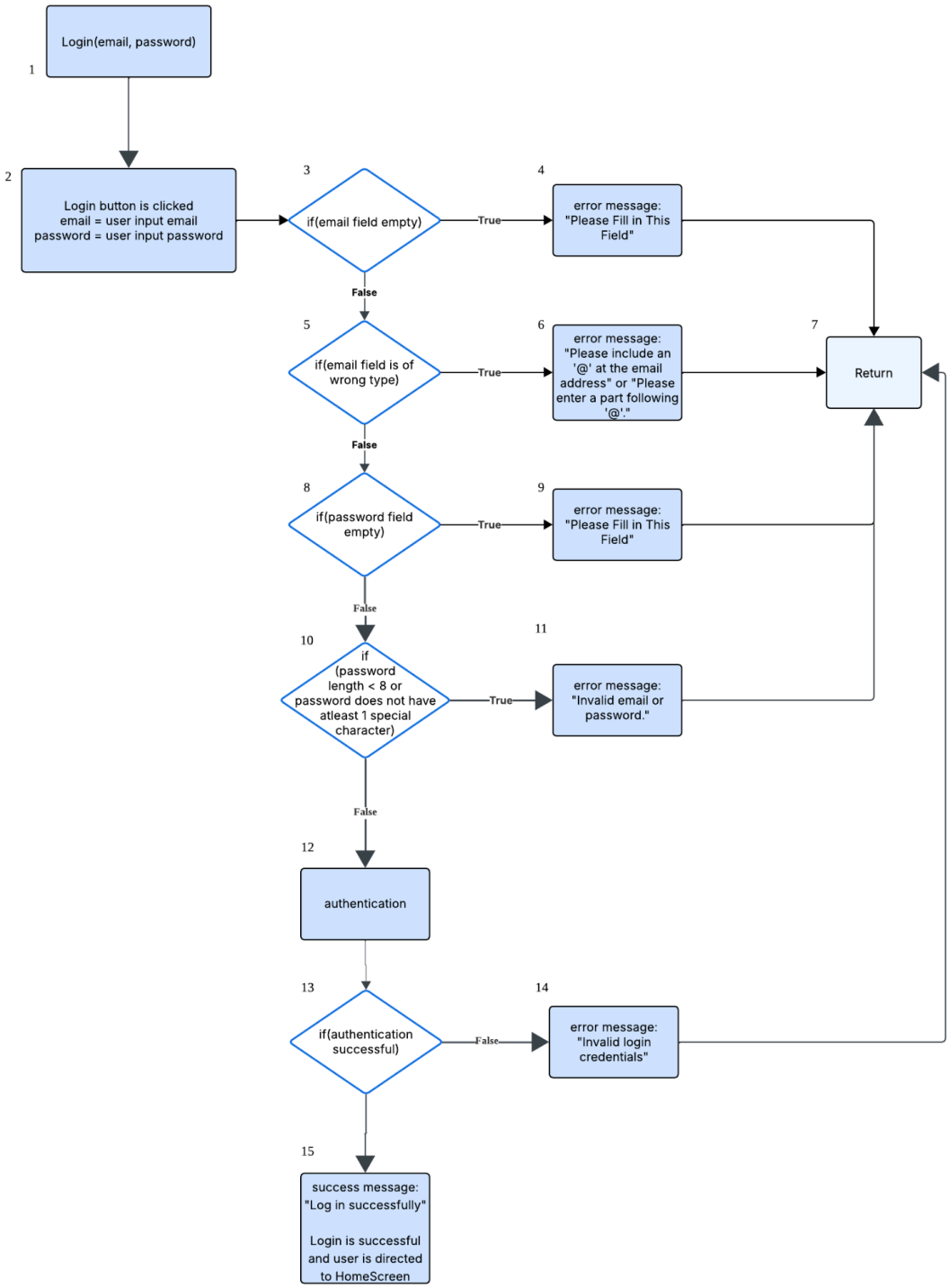
3	Empty Password email="testuser@gmail.com" username = "testuser" password=""	"Please Fill in This Field"	"Please Fill in This Field"	Yes
4	Empty Email email="" username = "testuser" password="password1234@"	"Please Fill in This Field"	"Please Fill in This Field"	Yes
3	Invalid Email Format 1 email="testuser" username = "testuser" password="password1234@"	"Please include an '@' at the email address."	"Please include an '@' at the email address."	Yes
4	Invalid Email Format 2 email="testuser@" username = "testuser" password="password1234@"	"Please enter the part following '@'."	"Please enter the part following '@'."	Yes
6	Invalid Password Format 1 email="testuser@gmail.com" username = "testuser" password="pass"	"Password must be at least 8 characters long!"	"Password must beat least 8 characters long!"	Yes
7	Invalid Password Format 2 email="testuser@gmail.com" username = "testuser" password="password1234"	"Password must have at least 1 special character!"	"Password must have at least 1 special character!"	Yes

8	Email In Use email="existingemail@gmail.com" username = "testuser" password="password1234@"	"Email in use, choose another email"	"Email in use, choose another email"	Yes
9	Username in Use email="testuser@gmail.com" username = "existingusername" password="password1234@"	"Username in use, choose another username"	"Username in use, choose another username"	Yes

10.2 White Box testing

10.2.1 Login

10.2.1.1 Control Flow Graph



(For a clearer view of the image, please refer to the PNG attached in the GitHub repository.)

10.2.1.2 Basic Path Testing

Cyclomatic Complexity: | Decision points | +1 = 5 + 1 = 6

Basic Path:

1. Baseline Path (All valid): 1, 2, 3, 5, 8, 10, 12, 13, 15
2. Basis Path 2 (Email empty field): 1, 2, 3, 4, 7
3. Basis Path 3 (Wrong email field): 1, 2, 3, 5, 6, 7
4. Basis Path 4 (Password empty field): 1, 2, 3, 5, 8, 9, 7
5. Basis Path 5 (Invalid password length / no special character): 1, 2, 3, 5, 8, 10, 11, 7
6. Basis Path 6 (Invalid login credentials): 1, 2, 3, 5, 8, 10, 12, 13, 14, 7

10.2.1.3 Test Cases and Results

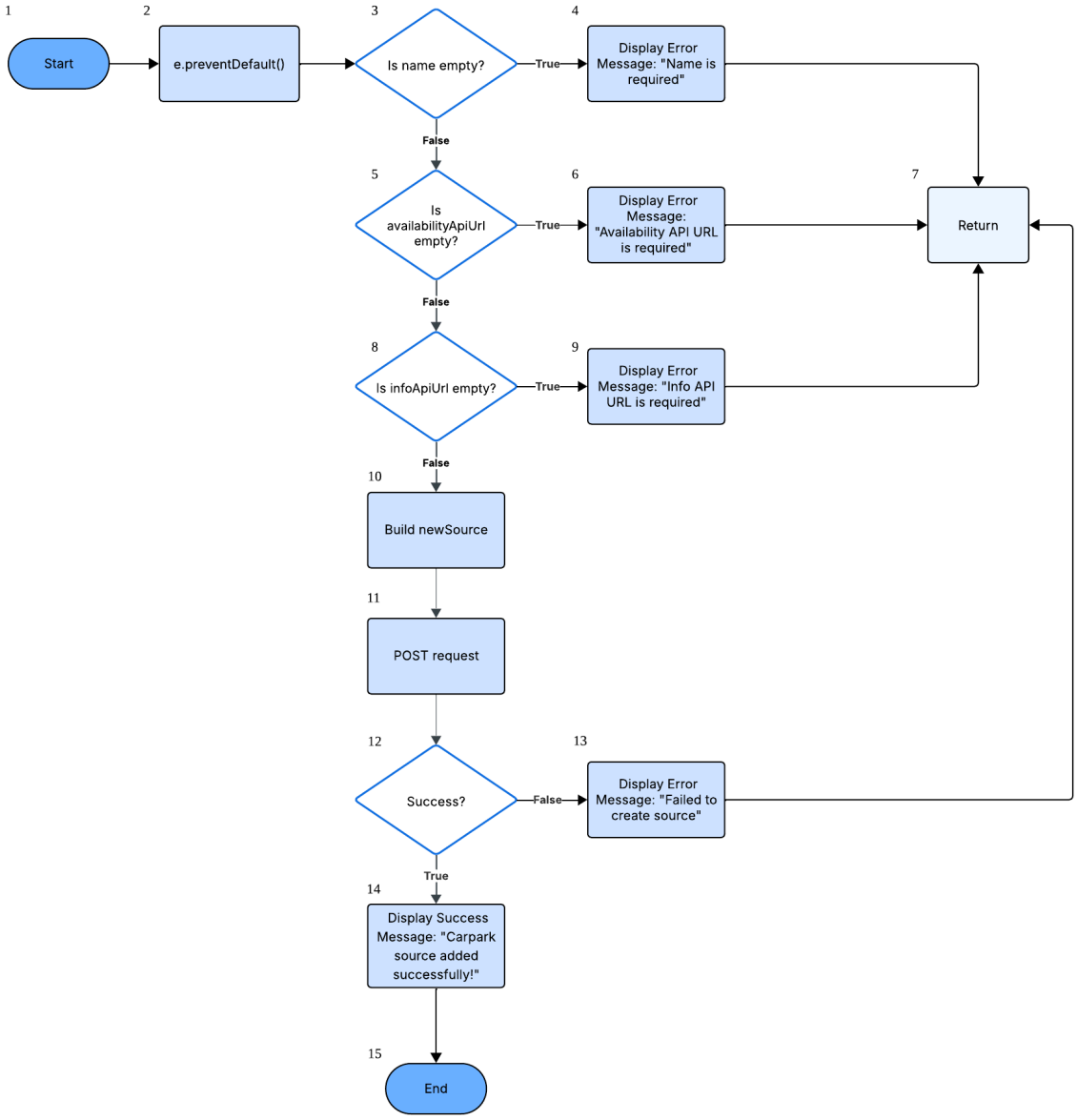
No.	Test Input	Expected Output	Actual Output	Pass?
1	email="test@gmail.com" password="password1234@"	"Log in successfully"	"Log in successfully"	Yes
2	email="" password="password1234@"	"Please Fill in This Field"	"Please Fill in This Field"	Yes
3	email="test" password="password1234@"	"Please include an '@' at the email address."	"Please include an '@' at the email address."	Yes
4	email="test@" password="password1234@"	"Please enter the part following '@'."	"Please enter the part following '@'."	Yes

5	email=“test@gmail.com” password=“”	"Please Fill in This Field."	"Please Fill in This Field."	Yes
6	email=“test@gmail.com” password=“pass”	"Invalid email or password."	"Invalid email or password."	Yes
	email=“test@gmail.com” password=“password1234”	"Invalid email or password."	"Invalid email or password."	Yes
7	email=“nonuser@gmail.com” password=“password1234@” *Note: Valid format but test case represent a non-registered user	“Invalid login credentials”	“Invalid login credentials”	Yes

10.2.2 AddCarparkSource

10.2.2.1 Control Flow Graph

AddCarparkSource



(For a clearer view of the image, please refer to the PNG attached in the GitHub repository.)

10.2.2.2 Basic Path Testing

Cyclomatic Complexity: | Decision points | + 1 = 4 + 1 = 5

Basic Path:

7. Baseline Path (All valid): 1, 2, 3, 5, 8, 10, 11, 12, 14, 15
8. Basis Path 2 (Missing name): 1, 2, 3, 4, 7
9. Basis Path 3 (Missing availabilityApiUrl): 1, 2, 3, 5, 6, 7
10. Basis Path 4 (Missing infoApiUrl): 1, 2, 3, 5, 8, 9, 7
11. Basis Path 5 (All valid, POST fails): 1, 2, 3, 5, 8, 10, 11, 12, 13, 15

Test Cases and Results:

Function: handleSubmit(e)
Inputs: name, availabilityApiUrl, infoApiUrl

No.	Test Input	Expected Output	Actual Output	Pass?
1	name = "HDB Carparks" availabilityApiUrl = "https://api.data.gov.sg/carpark_availability" infoApiUrl = "https://api.data.gov.sg/carpark_info"	Display success message: "Carpark source added successfully!"	Display success message: "Carpark source added successfully!"	Yes
2	name = " " availabilityApiUrl = "https://api.data.gov.sg/carpark_availability" infoApiUrl = "https://api.data.gov.sg/carpark_info"	Display error message: "Name required." Return (no POST request sent)	Display error message: "Name required." Return (no POST request sent)	Yes
3	name = "HDB Carparks" availabilityApiUrl = " " infoApiUrl = "https://api.data.gov.sg/carpark_info"	Display error message: "Availability URL required." Return (no POST request sent)	Display error message: "Availability URL required." Return (no POST request sent)	Yes

4	name = “HDB Carparks” availabilityApiUrl = “https://api.data.gov.sg/carpark_availability” infoApiUrl = “ ”	Display error message: “Info URL required.” Return (no POST request sent)	Display error message: “Info URL required.” Return (no POST request sent)	Yes
5	name = “HDB Carparks” availabilityApiUrl = “https://invalid-url.example” infoApiUrl = “https://api.data.gov.sg/carpark_info”	HTTP 400 or network error Display error message: “Failed to add source.”	HTTP 400 error returned Display error message: “Failed to add source.”	Yes

11. Demo Script

Presentation slides attached in github

Live Demo

Feature Showcase:

1) Login and Registration
2) Carpark Availability
    a) Show map (Navigating around and clicking on markers)
    b) Filters
    c) Show carpark detail
3) External Api Parsing (Carpark Source)
    a) Show the creation of carpark sources that have different json format from the data.gov.sg
4) Bus
    a) Show bus arrival timing
    b) Show the bus stop location on map (nearby location)
    c) Search bus stops / bus number to find arrival timing
    d) Shows seat availability
5) Mrt
    a) Shows MRT location in map + surrounding MRTs
    b) MRT lines are colour-coded (e.g. Circle line- yellow, Downtown line- blue)
    c) Crowd density per MRT station (Low, Medium, High)

References

1. *Real-time carpark availability in Singapore*. CarparkSG. (n.d.). https://carparksg.com/
2. Google. (n.d.). Google maps. https://www.google.com/maps